Universität Konstanz

# Introduction to Computing for the Social Sciences

## Session 7 – Sorting Algorithms

David Garcia

Winter Semester 2026-2027

# Course Outline

- Session 1: Introduction
- Session 2: Information Coding
- Session 3: Data Structures
- Session 4: Software
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Time Complexity
- Session 9: Theoretical Computer Science
- Session 10: Machine Learning (8.01)

  K503, lecture 13:30-15:00, practice 15:15-16:45
- Session 11: Databases
- Session 12: Parallel Programming
- Session 13: Social Science Applications
- Session 14: Exam Review

# Outline

1. Sorting
2. Iterative sorting: Selection sort
3. Divide and Conquer: Mergesort and Quicksort
4. Data structures for sorting: Heapsort

# Sorted by region, then by age, and the regions came apart

Universität Konstanz

**MOTIVATING EXAMPLE**

You sort a respondent table by **region**, then sort the same table by **age** to inspect the age distribution. The regions, which were in a deliberate order, come back shuffled, because the second sort was allowed to move equal-aged rows past one another. Nothing warned you. The appendix tables are wrong.

- Both sorts produced a **correctly sorted** table.
- Only one of them preserved the work of the first.

**Two algorithms, the same sorted order, different results. What property separates them, and which do you have?**

# Two seconds, or fourteen hours

Universität Konstanz

**MOTIVATING EXAMPLE**

A million records. **Selection sort** compares every element against every other: about **$5 \times 10^{11}$** comparisons. **Mergesort** halves the problem repeatedly: about **$2 \times 10^{7}$**. At ten million comparisons a second, that is the difference between **fourteen hours** and **two seconds**.

**$5 \times 10^{11}$**
comparisons, selection sort

**$2 \times 10^{7}$**
comparisons, mergesort

**25 000×**
fewer, same result

**Both are correct. What is it about the *shape* of one procedure that makes it collapse at scale?**

# Why sort at all?

Universität Konstanz

**MOTIVATING EXAMPLE**

A codebook lookup costs **13** comparisons if the list is sorted and **6 000** if it is not. But sorting the codebook costs something too. With **one** lookup, sorting is wasted effort. With **forty million**, it is the only thing that makes the job possible.

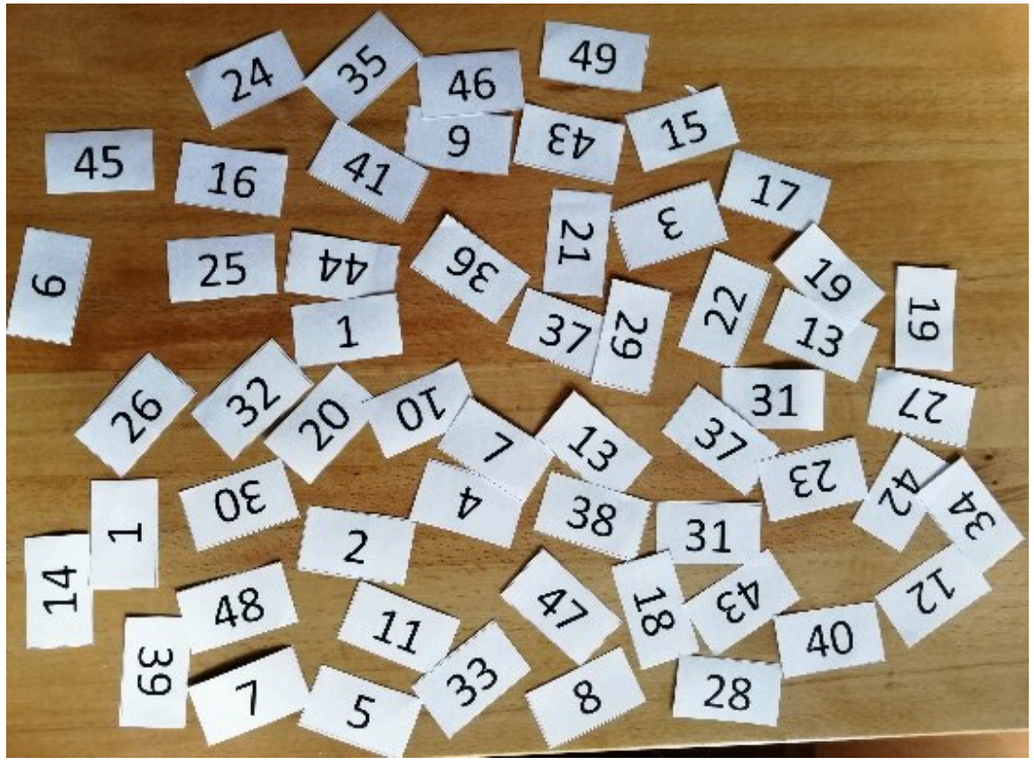
**1**
lookup: don’t sort, you would pay more than you save

**25**
lookups: the break-even point

**$40 \times 10^6$**
lookups: sorting is the whole difference

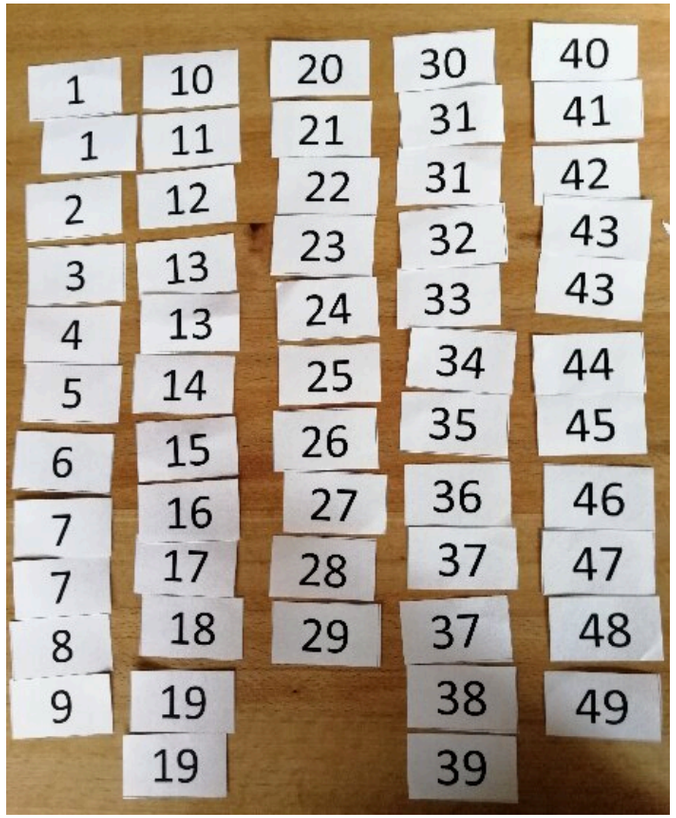
**Sorting is an investment. How do you work out whether it pays?**

# Sorting




**Which numbers are duplicates?**

# Sorting

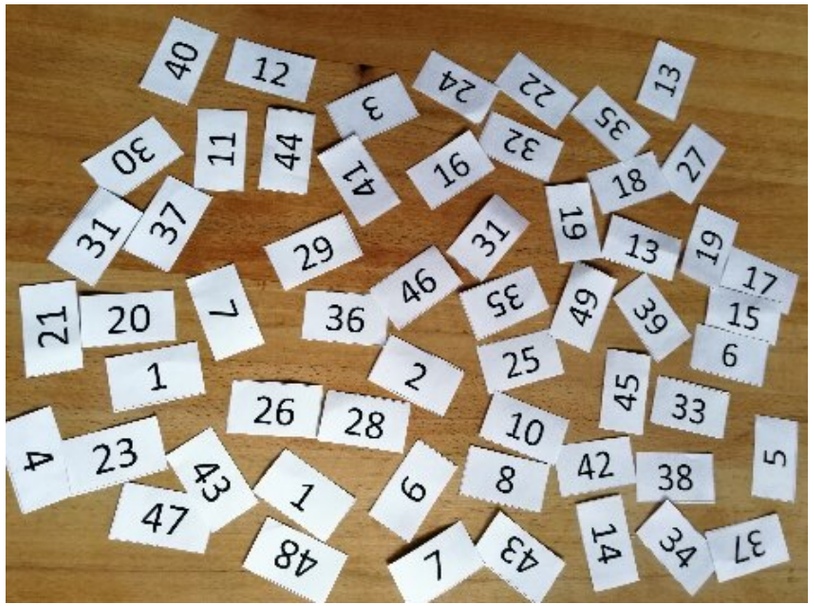


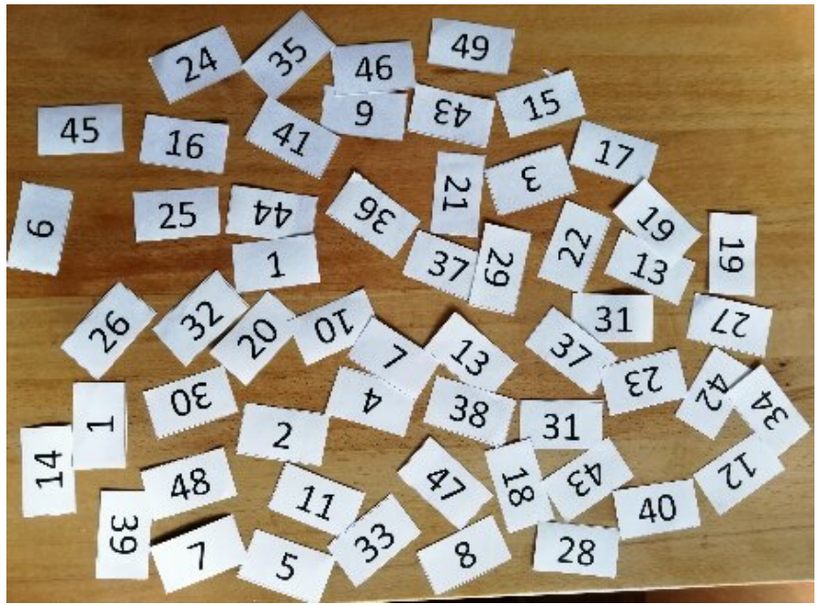

**Which numbers are duplicates?**

# Sorting

Universität Konstanz

## Are the number sets different?

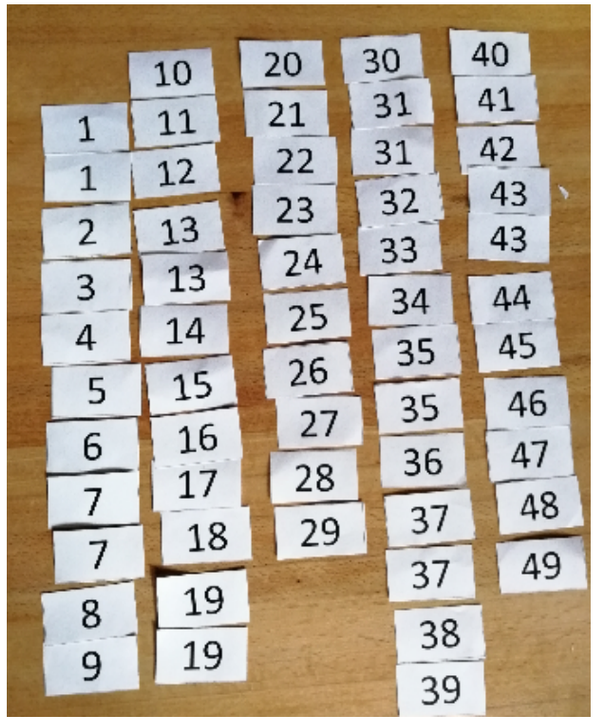


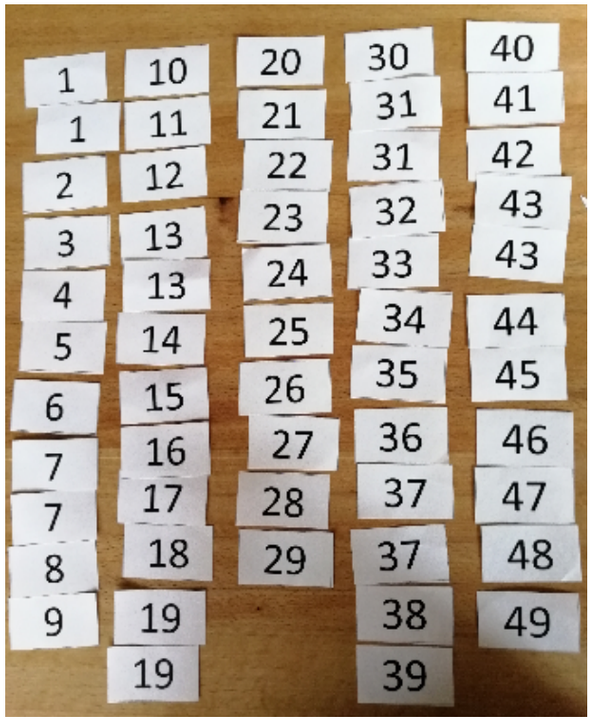



# Sorting

Universität Konstanz

## Are the number sets different?

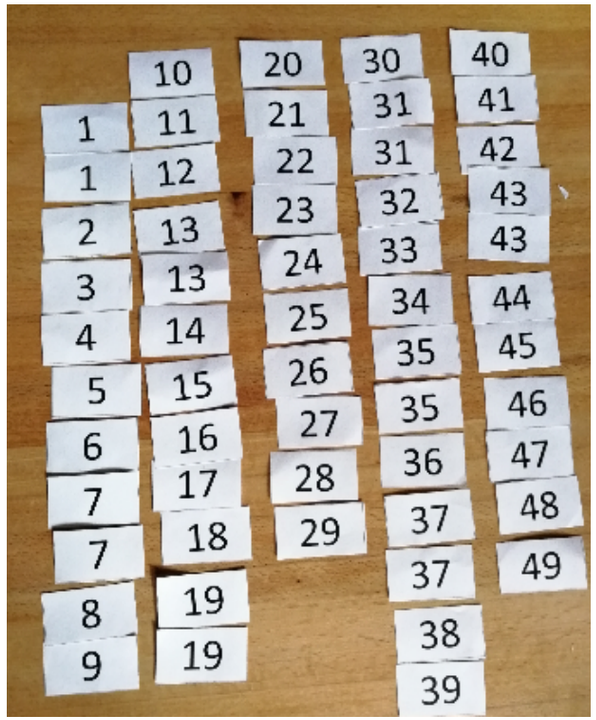


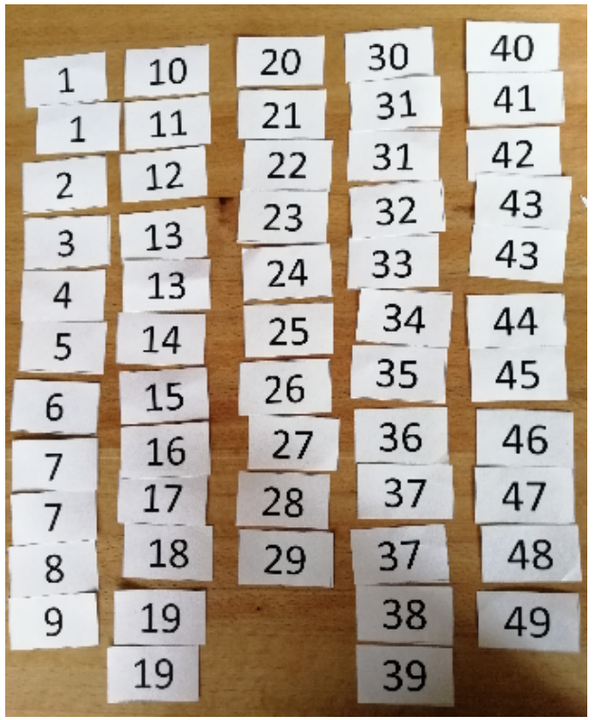



# Sorting

Universität Konstanz

## Are the number sets different?



- **Fundamentals**
  - we are operating on:
    - set of objects $U$
    - set of keys $S$ with linear order relation $\leq$ (i.e., a linearly ordered set)
    - sorting function that maps all objects $U$ to $S$ (i.e., $s : U \rightarrow S$)
  - we are searching for a permutation of indices $i_1 \ldots i_n$ of the objects in $U$, such that:

$$\forall\, 1 \leq k \leq n-1 : \;\; s(u_{i_k}) \leq s(u_{i_{k+1}})$$

# Stable vs. unstable sorting

Universität Konstanz

- **Stable sorting methods**
  - different objects can have the same key value used for sorting, e.g., students with the exact same name
  - If a sorting algorithm, after sorting the contents, does not change the sequence of similar content in which they appear, it is called **stable** sorting. Otherwise it is **unstable** sorting.
  - stable sorting methods ensure that
    - objects with the same value for a given sorting function $s_1$ have different values for another function $s_2$
    - sorting according to $s_1$ in this case does not change the order of the objects

**example: sort by first name**

| | |
|---|---|
| 123456 | Christina |
| 234567 | Sandra |
| 345678 | Sandra |
| 456789 | Carl |

| | |
|---|---|
| 456789 | Carl |
| 123456 | Christina |
| 234567 | Sandra |
| 345678 | Sandra |

**stable**

| | |
|---|---|
| 456789 | Carl |
| 123456 | Christina |
| 345678 | Sandra |
| 234567 | Sandra |

**unstable**

# Iterative sorting



1. Sorting
2. **Iterative sorting: Selection sort**
3. Divide and Conquer: Mergesort and Quicksort
4. Data structures for sorting: Heapsort

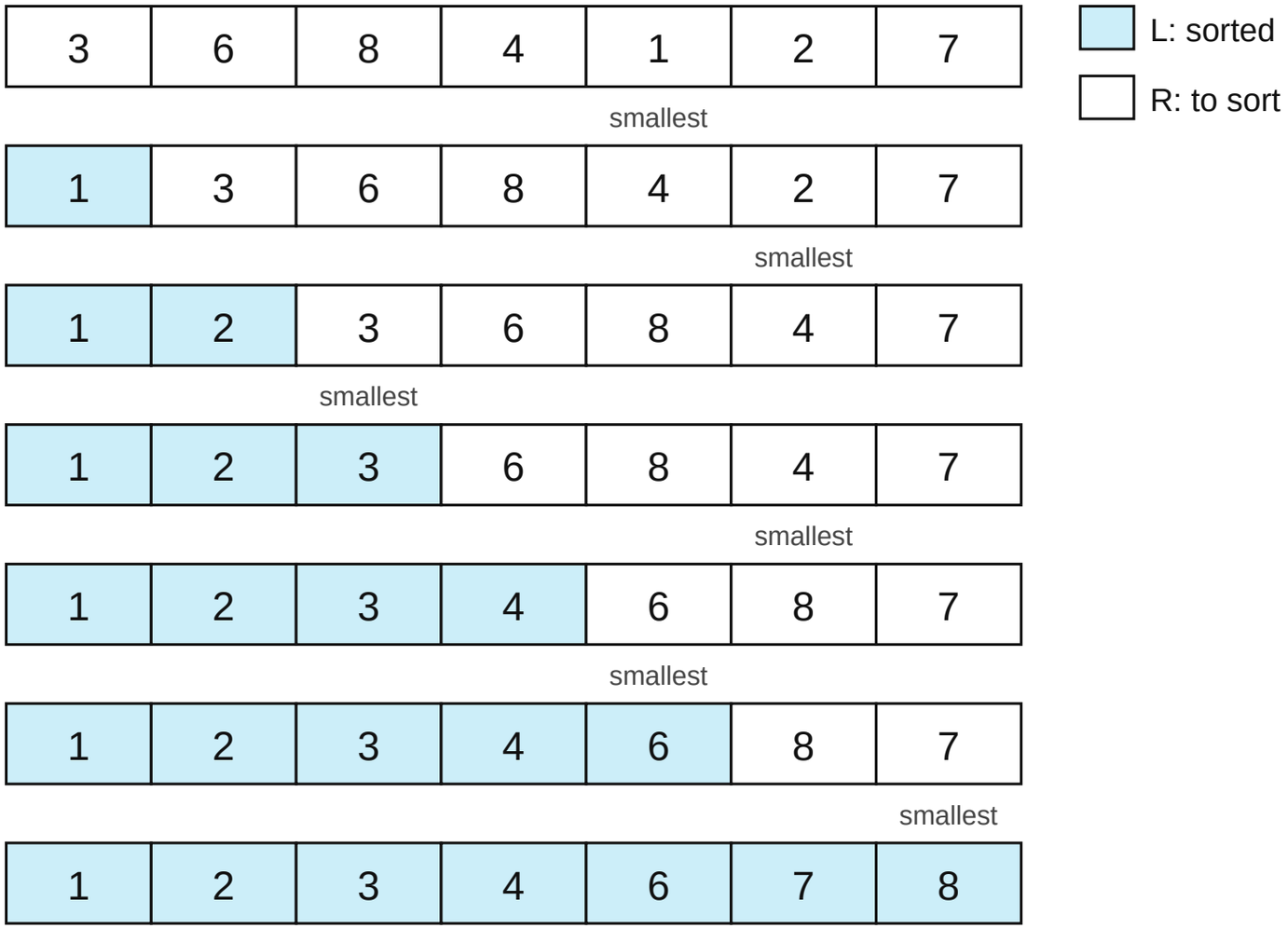
# Iterative Sorting: Repeated Minimum Search



**Sorting through selection as fundamental principle**

- intuitive strategy:
  - find maximum/minimum of a set and move it to front
  - iteratively apply the logic to the remaining set
- algorithmic strategy (**repeated minimum search**)
  - split list into two parts
    - sorted part on left, $L$
    - unsorted part on right, $R$
  - at start: sorted part is empty, i.e. $L = \emptyset$, unsorted part $R$ is the entire list
  - find minimum in unsorted $R$ and append to sorted $L$
  - stop if $\text{length}(R) = 1$

# Repeated minimum search



# Selection sort



**Procedure** SELECTION-SORT(*A*, *n*)

*Inputs:*

*A*: an array

*n*: the number of elements in *A* to sort

*Result:* the elements of *A* are sorted into nondecreasing order

**For** *i* = 1 to *n* - 1:

    *smallest* ← *i*

    **For** *j* = *i* + 1 to *n*:

        **If** *A*[*j*] < *A*[*smallest*]

            **then** *smallest* ← *j*

    swap *A*[*i*] with *A*[*smallest*]

From *Algorithms Unlocked*. Thomas Cormen, 2013, pp. 32–34

# Selection sort

Universität Konstanz

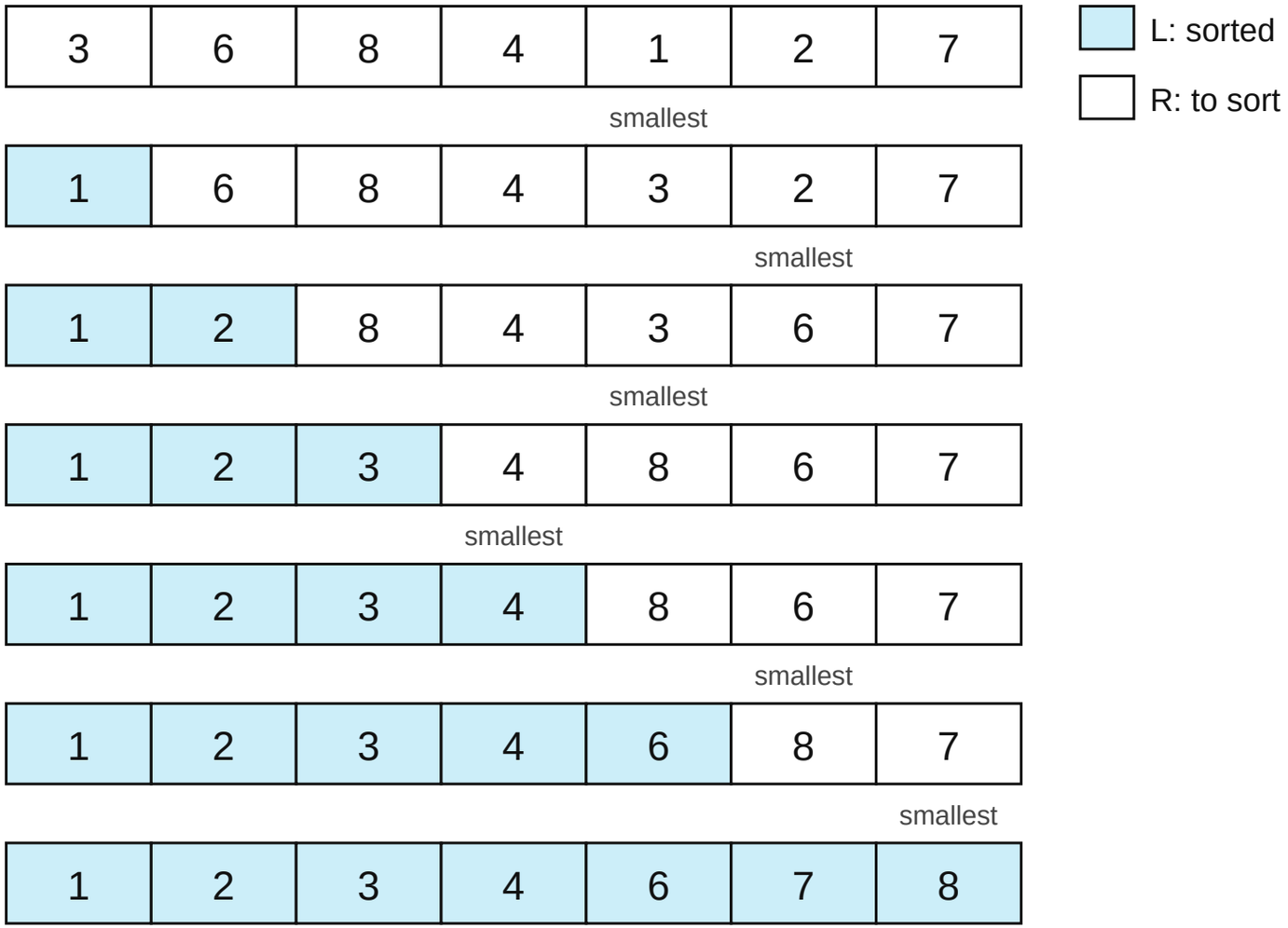
**Procedure** SELECTION-SORT(*A*, *n*)
*Inputs:*
*A*: an array
*n*: the number of elements in *A* to sort
*Result:* the elements of *A* are sorted into nondecreasing order
**For** *i* = 1 to *n* - 1:
    *smallest* ← *i*
    **For** *j* = *i* + 1 to *n*:
        **If** *A*[*j*] < *A*[*smallest*]
            **then** *smallest* ← *j*
    swap *A*[*i*] with *A*[*smallest*]

**Find the smallest in R**

**Swap it to be the last of L**

From *Algorithms Unlocked*. Thomas Cormen, 2013, pp. 32–34

# Selection sort example



# Repeated minimum search vs selection sort

Universität Konstanz

**Characteristics of iterative sorting algorithms**

- both algorithms are **partially correct**
  - any element still in $R$ is larger or equal to any in $L$, $L$ is sorted
- both algorithms **terminate**
  - remaining part $R$ shrinks in every step, eventually it has to have length 1
- different use depending on data structure of list:
  - repeated minimum appends in the middle of the list: better for **linked lists**
  - selection sort uses swaps: better for **arrays**
- **stability**:
  - repeated minimum is *stable*:
    - in case of ambiguity, takes the first minimum in $R$
    - this always correctly preserves any other ordering
  - selection sort is *unstable*:
    - swapping does not necessarily preserve other ordering in $R$

# Outtake – Social Science Applications

Universität Konstanz

**Why should we care about sorting?**

- Sorting for the sake of sorting
  - we are sorting data more often than we think
  - especially for data science applications complex data often needs to be sorted to be analyzed or shown
  - in practice, speed matters here for analysts and users
  - sorting is healthy: it helps to notice data issues
- Sorting for searching
  - many search algorithms require sorted sets of data to work (e.g., binary search)
  - it is in practice much easier to find items in an ordered than an unordered list

**Exercise: write the pseudocode of repeated minimum search**

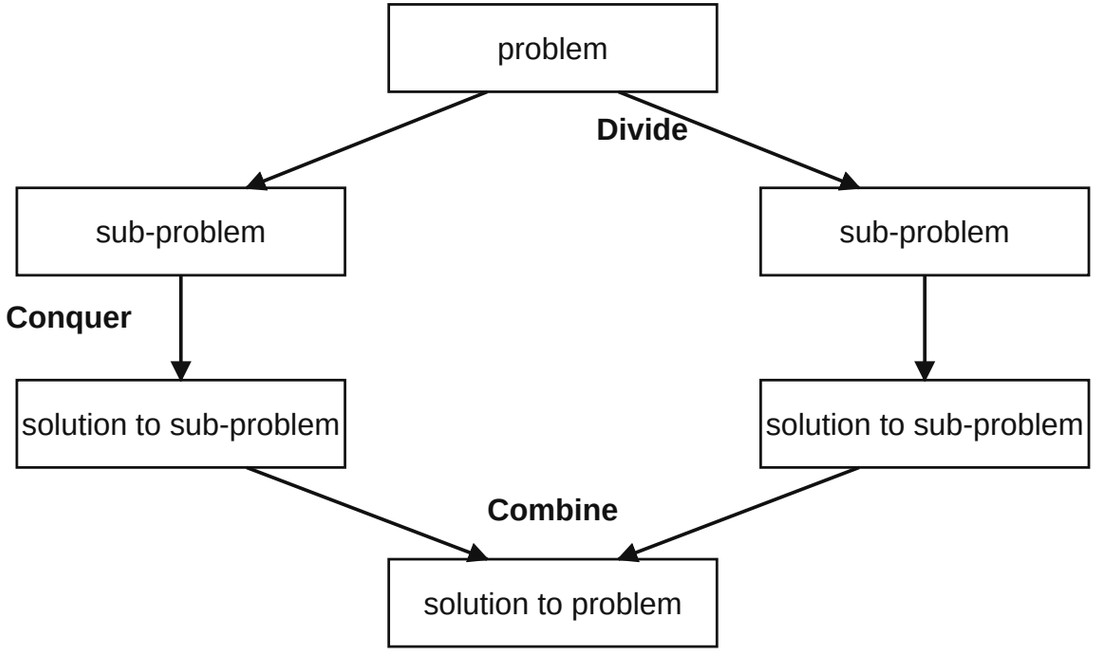
# Divide and Conquer

Universität Konstanz

1. Sorting
2. Iterative sorting: Selection sort
3. **Divide and Conquer: Mergesort and Quicksort**
4. Data structures for sorting: Heapsort

# Divide and Conquer

- Fundamental algorithmic principle inspired by military history: *divide et impera*
  - separate a problem into separate sub-problems
  - continue until those sub-problems are solvable
  - then recombine partial results to full solution



# Divide and Conquer



**Divide and conquer $\neq$ recursion**

- it is not the same as recursion, i.e., recursive calls of the same function
- recursion is a natural implementation strategy for algorithms following a divide and conquer approach
  - by definition, the sub-problems in divide and conquer are self-similar
  - generate overall solution by combining solutions of sub-problems
- BUT we can also implement a divide and conquer strategy iteratively

# Mergesort

Universität Konstanz

- **Mergesort** first divides the list into equal halves and then combines them in a sorted manner. Mergesort sorts by *merging* (sub-)lists
- principle:
  - a list with one element is trivially sorted
  - merging two sorted lists is simple:
    - compare first element of each sub-list
    - append smaller of the two to merged list and remove from sub-list
    - continue with remaining sub-lists
- algorithmic strategy
  - repeated division of list until just one element is remaining per list
  - repeated merging of sorted sub-lists until only one sorted list remains

# Mergesort

**Procedure** MERGE-SORT($A$, $p$, $r$)

*Inputs:*

$A$: an array

$p$, $r$: the starting and ending index of the subarray

*Result:* the elements of the subarray $A[p..r]$ are sorted into nondecreasing order

**If** $p \geq r$

**then** return without doing anything # A[p..r] has at most one element

**else** $q \leftarrow \lfloor (p + r) / 2 \rfloor$

MERGE-SORT($A$, $p$, $q$) # sort the left half recursively

MERGE-SORT($A$, $q + 1$, $r$) # sort the right half recursively

MERGE($A$, $p$, $q$, $r$) # merge the two sorted halves

From *Algorithms Unlocked*. Thomas Cormen, 2013, pp. 40–47

# The MERGE function



**Procedure** MERGE($A$, $p$, $q$, $r$)

*Inputs:*

$A$: an array

$p$, $q$, $r$: indices such that $A[p..q]$ and $A[q+1..r]$ are sorted

*Result:* $A[p..r]$ contains the elements of both subarrays, sorted

$n_1 \leftarrow q - p + 1$; $n_2 \leftarrow r - q$

$B[1..n_1] \leftarrow$ copy of $A[p..q]$; $C[1..n_2] \leftarrow$ copy of $A[q+1..r]$

$B[n_1 + 1] \leftarrow \infty$; $C[n_2 + 1] \leftarrow \infty$ # sentinels, never copied

$i \leftarrow 1$; $j \leftarrow 1$

**For** $k = p$ to $r$:

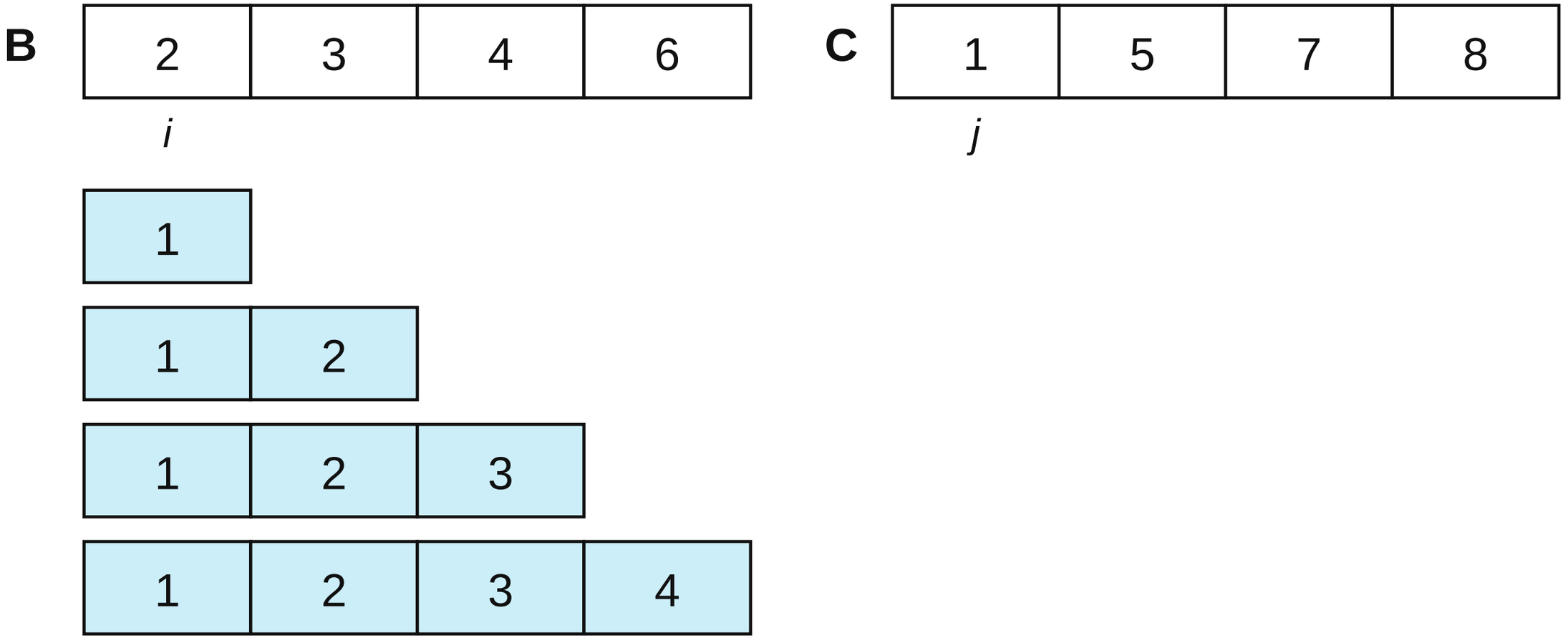
**If** $B[i] \leq C[j]$

**then** $A[k] \leftarrow B[i]$; $i \leftarrow i + 1$

**else** $A[k] \leftarrow C[j]$; $j \leftarrow j + 1$

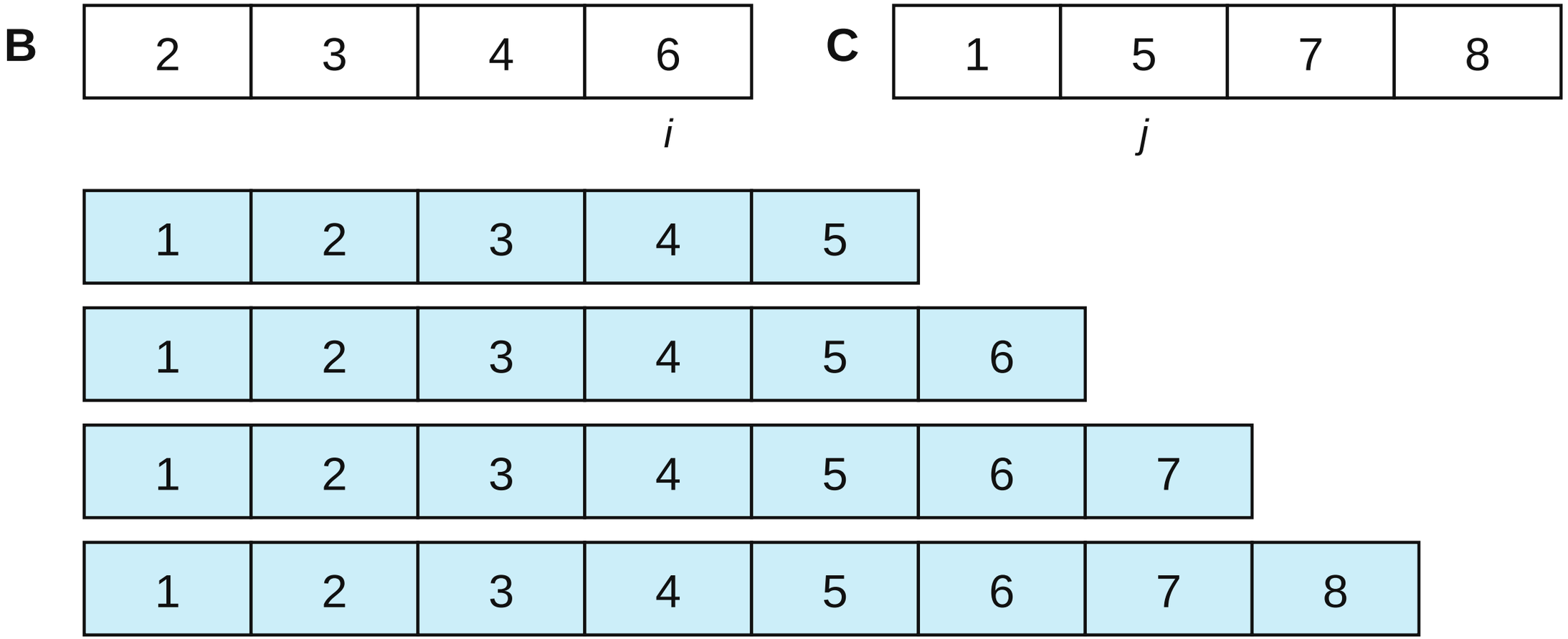
# MERGE example





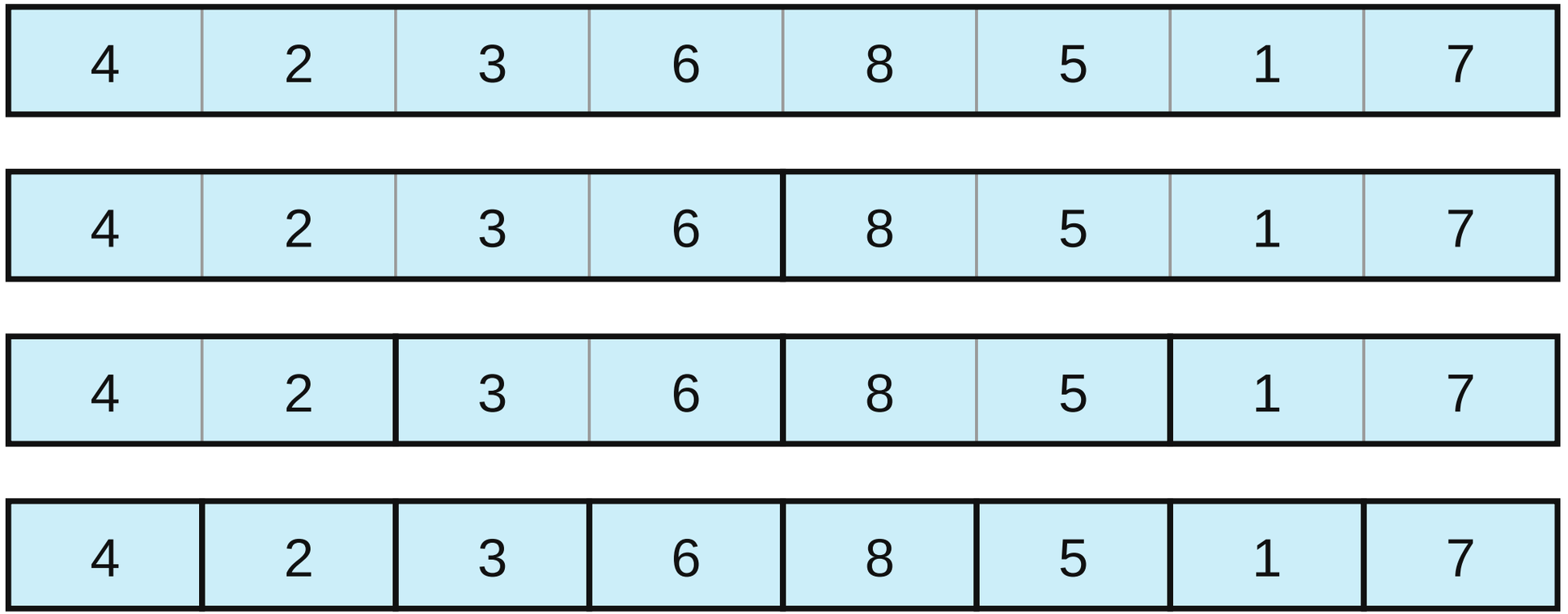
# MERGE example





# Mergesort

- **Mergesort split:** first divides the whole list iteratively into equal halves (no change in sequence of items in the original) until atomic values are achieved.



# Mergesort

- **Mergesort algorithm** – merge them into another list in a sorted manner

| 4 | 2 | 3 | 6 | 8 | 5 | 1 | 7 |
|---|---|---|---|---|---|---|---|

| 2 | 4 | 3 | 6 | 5 | 8 | 1 | 7 |
|---|---|---|---|---|---|---|---|

# Mergesort

- **Mergesort algorithm** – merge them into another list in a sorted manner

| 4 | 2 | 3 | 6 | 8 | 5 | 1 | 7 |
|---|---|---|---|---|---|---|---|

| 2 | 4 | 3 | 6 | 5 | 8 | 1 | 7 |
|---|---|---|---|---|---|---|---|

| 2 | 3 | 4 | 6 | 1 | 5 | 7 | 8 |
|---|---|---|---|---|---|---|---|

# Mergesort

- **Mergesort algorithm** – merge them into another list in a sorted manner

| 4 | 2 | 3 | 6 | 8 | 5 | 1 | 7 |
|---|---|---|---|---|---|---|---|

| 2 | 4 | 3 | 6 | 5 | 8 | 1 | 7 |
|---|---|---|---|---|---|---|---|

| 2 | 3 | 4 | 6 | 1 | 5 | 7 | 8 |
|---|---|---|---|---|---|---|---|

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|

## Mergesort as a divide and conquer strategy

- **Divide:** splitting into two sub-lists (with half the size)
  - recursive implementation of sub-list sorting
    - call mergesort on itself
    - assume that output is a sorted list in each recursion step
    - requires well-defined base case (= list with one element)
- **Combine:** merging two lists preserving order
  - systematic and efficient merger of partial lists to one large sorted list
  - see *Algorithms Unlocked* (Cormen, 2013) for MERGE pseudocode

# Mergesort

Universität Konstanz

## Characteristics of mergesort

- **correctness**
  - merge steps sort correctly
  - no changing of order outside of merging
  - all elements are sorted and algorithm terminates
- **memory and stability**
  - need additional memory (in addition to list) for handling sub-lists
  - stable algorithm because other ordering structure is preserved during merging

# John von Neumann: more than mergesort's father

**John von Neumann** (Neumann János Lajos, Budapest, 1903-1957)



- One of the most influential figures of Computer Science, Math, and Physics in the 20th century
- Inventor of the mergesort algorithm
- Among the first to talk about the time complexity of algorithms in the 1950s
- Popularized the modern computer architecture with main memory, ALU, registries, etc.
- Proposed cellular automata that self-replicated
- Warned about anthropogenic global warming as early as 1955

# Quicksort



- Similar divide and conquer logic to mergesort but division into sub-lists not necessarily regular
- **Quicksort rationale:**
  - pick **pivot** element from list
  - sort list such that all elements left (right) of pivot are smaller (larger) than its value
  - after this **partition** operation the pivot element is already in its final (sorted) position, sorting now occurs only right/left of the pivot element
  - split list at pivot and apply logic recursively to left and right sub-list

# Quicksort



**Procedure** QUICK-SORT(*A*, *p*, *r*)

*Inputs and Result:* same as MERGE-SORT

**If** $p \geq r$

**then** return without doing anything

**else** $q \leftarrow$ PARTITION(*A*, *p*, *r*)  # A[q] is now in its final place

QUICK-SORT(*A*, *p*, *q* - 1)

QUICK-SORT(*A*, *q* + 1, *r*)

# The PARTITION function

*Procedure* PARTITION$(A, p, r)$

*Inputs:* Same as MERGE-SORT.

*Result:* Rearranges the elements of $A[p\,..\,r]$ so that every element in $A[p\,..\,q-1]$ is less than or equal to $A[q]$ and every element in $A[q+1\,..\,r]$ is greater than $q$. Returns the index $q$ to the caller.

1. Set $q$ to $p$.
2. For $u = p$ to $r-1$ do:
   A. If $A[u] \leq A[r]$, then swap $A[q]$ with $A[u]$ and then increment $q$.
3. Swap $A[q]$ with $A[r]$ and then return $q$.

**Exercise: run quicksort on pen and paper on the same example as mergesort**

# Quicksort: properties

- **advantages**
  - very memory efficient
  - in-place procedure: is (usually) faster than merge sort
- **disadvantages**
  - efficiency depends on choice of pivot and the partition scheme chosen (more next week)
- **differences from mergesort:**
  - in quicksort all the heavy lifting is done while dividing the array into subarrays, while in mergesort all the real work happens during merging the subarrays
  - in quicksort, the combine step does absolutely nothing

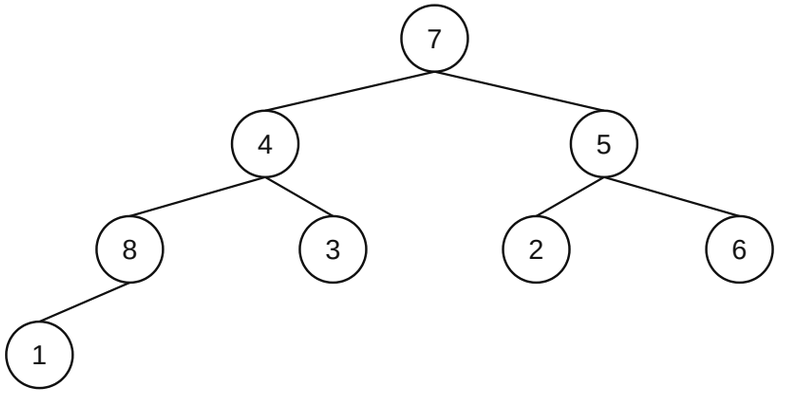
# Data structures for sorting



1. Sorting
2. Iterative sorting: Selection sort
3. Divide and Conquer: Mergesort and Quicksort
4. **Data structures for sorting: Heapsort**

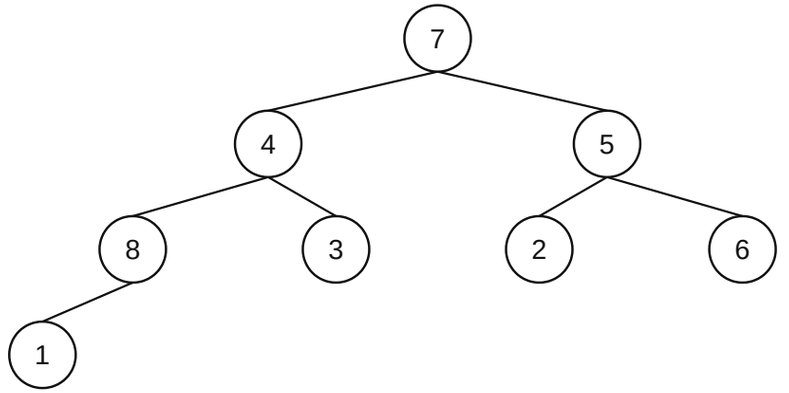
# Heaps

- Search algorithms can use data structures for help: **heaps** as an example
- partially ordered binary trees without gaps (or holes)
  - children of any node not larger than the node
  - not a binary search tree: only local ordering of nodes and children
  - all levels filled, the last one from the left
- example: does this tree satisfy the heap property?



# Heaps

- Search algorithms can use data structures for help: **heaps** as an example
- partially ordered binary trees without gaps (or holes)
  - children of any node not larger than the node
  - not a binary search tree: only local ordering of nodes and children
  - all levels filled, the last one from the left
- example: does this tree satisfy the heap property?

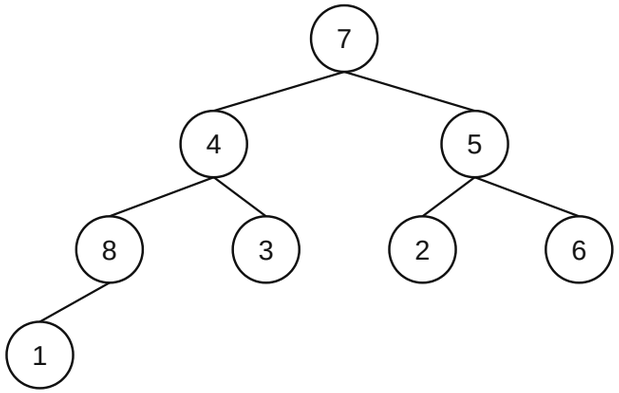


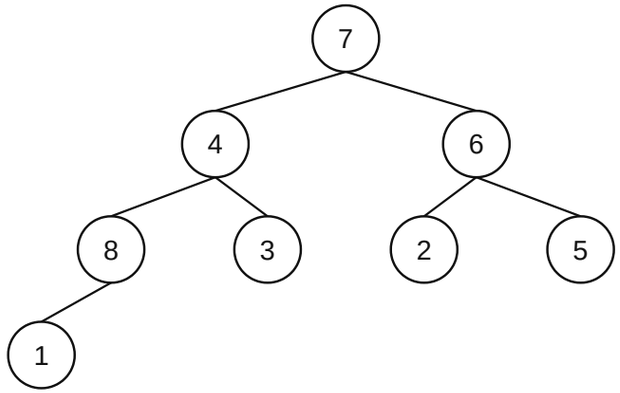

**No:** 8 is larger than its parent 4

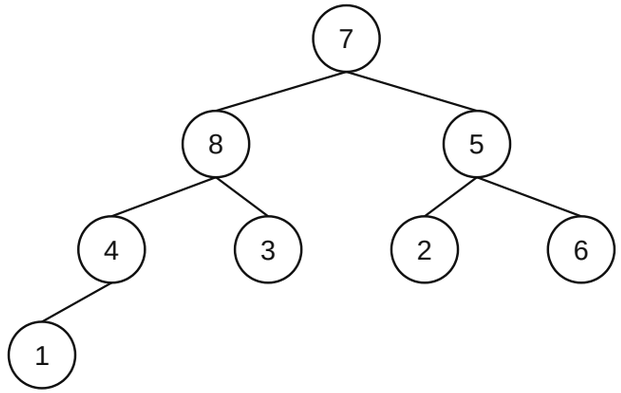
# Heaps

- partially ordered binary trees without gaps (or holes)
  - children of any node not larger than the node
  - not a binary search tree, i.e., just require local ordering of nodes and children. All levels filled, last filled from the left

- Heap property can be efficiently “recovered”

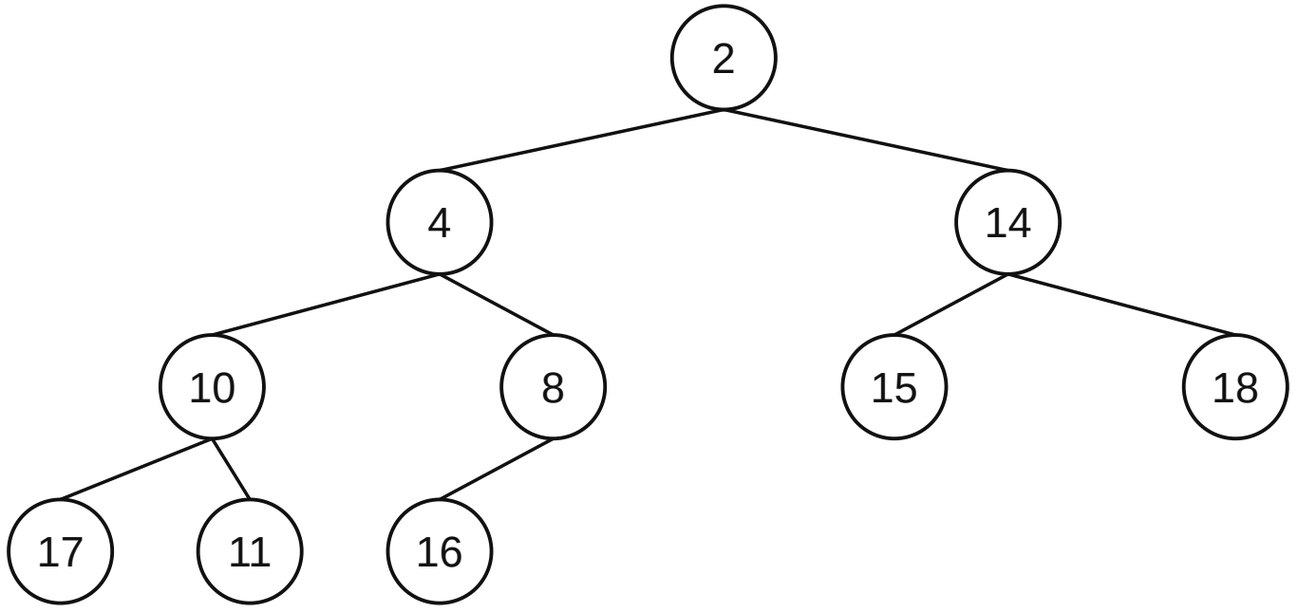


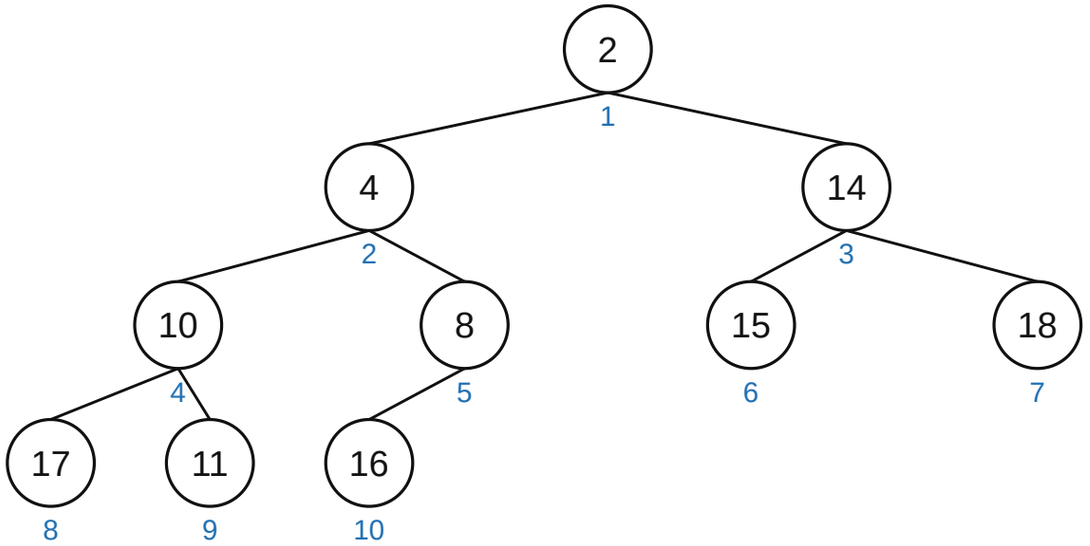
# Min-Heaps

- Same as a heap but:
  - children of any node not **smaller** than the node
  - All levels filled, last filled from the left



# Min-Heaps

- Same as a heap but:
  - children of any node not **smaller** than the node
  - All levels filled, last filled from the left




Min-heap as an array:

| 2 | 4 | 14 | 10 | 8 | 15 | 18 | 17 | 11 | 16 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

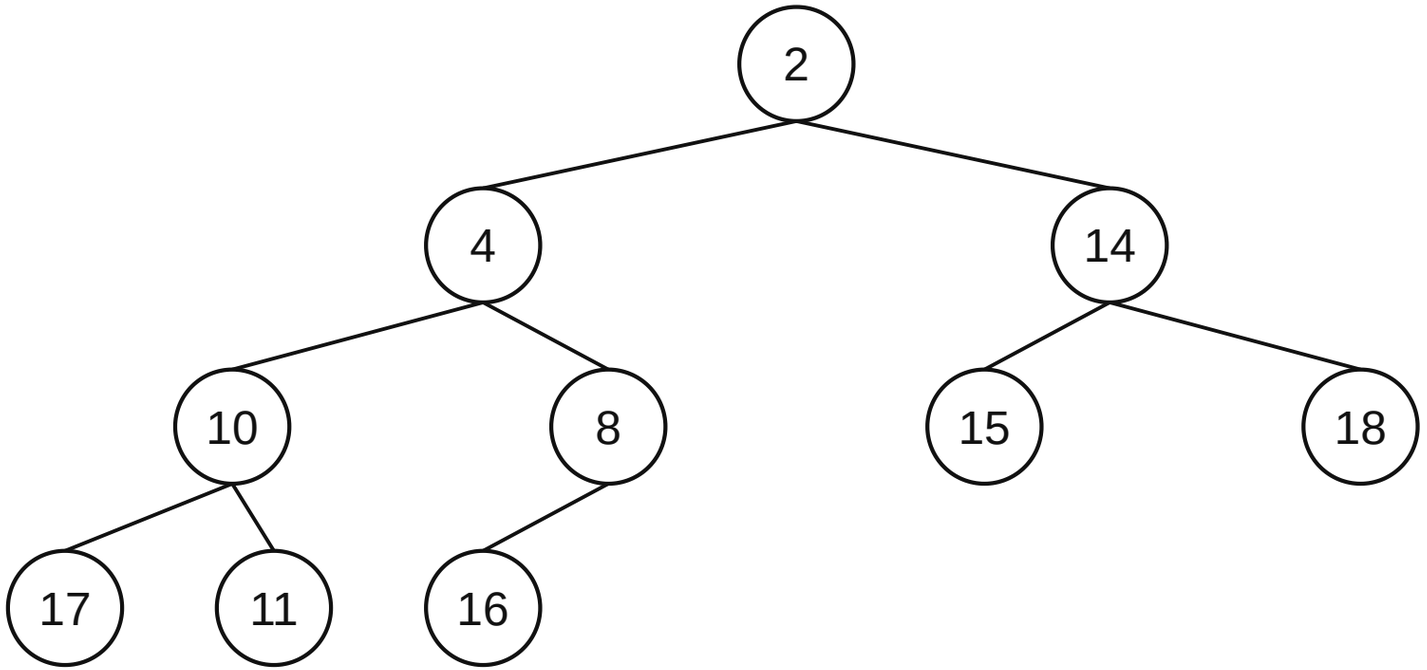
# Heapsort



- **conceptual foundation:**
  - in a min-heap the smallest element is on top
  - children are not smaller because of heap property
  - same is recursively true for the entire tree
  - we can start by building a heap of the list
- **idea:**
  - remove the top (smallest) element from tree
  - replace it with the last element of the tree
  - recover the heap property through permuting elements

# Heapsort: Example



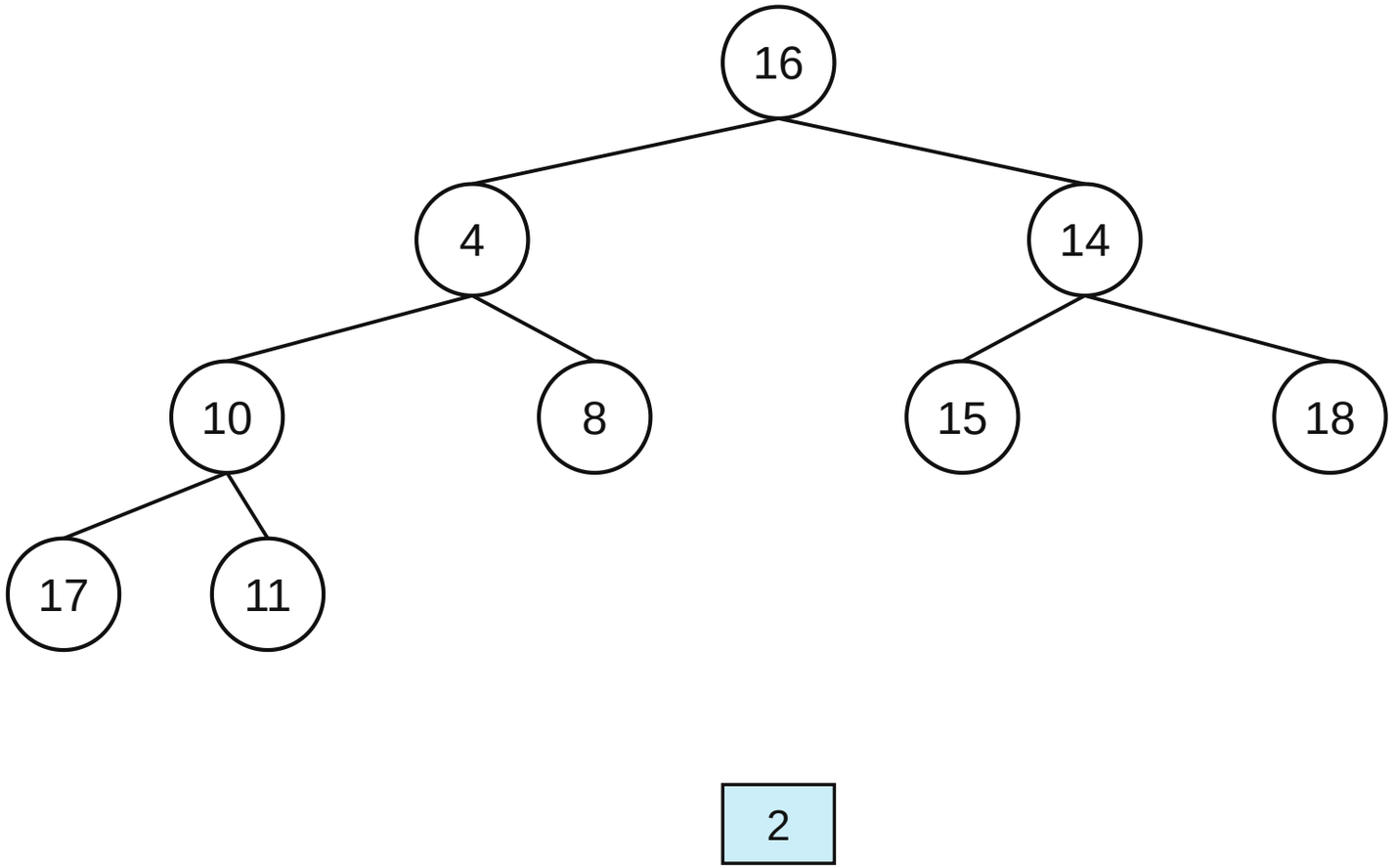



**Extract top (minimum) and place it first**

# Heapsort: Example

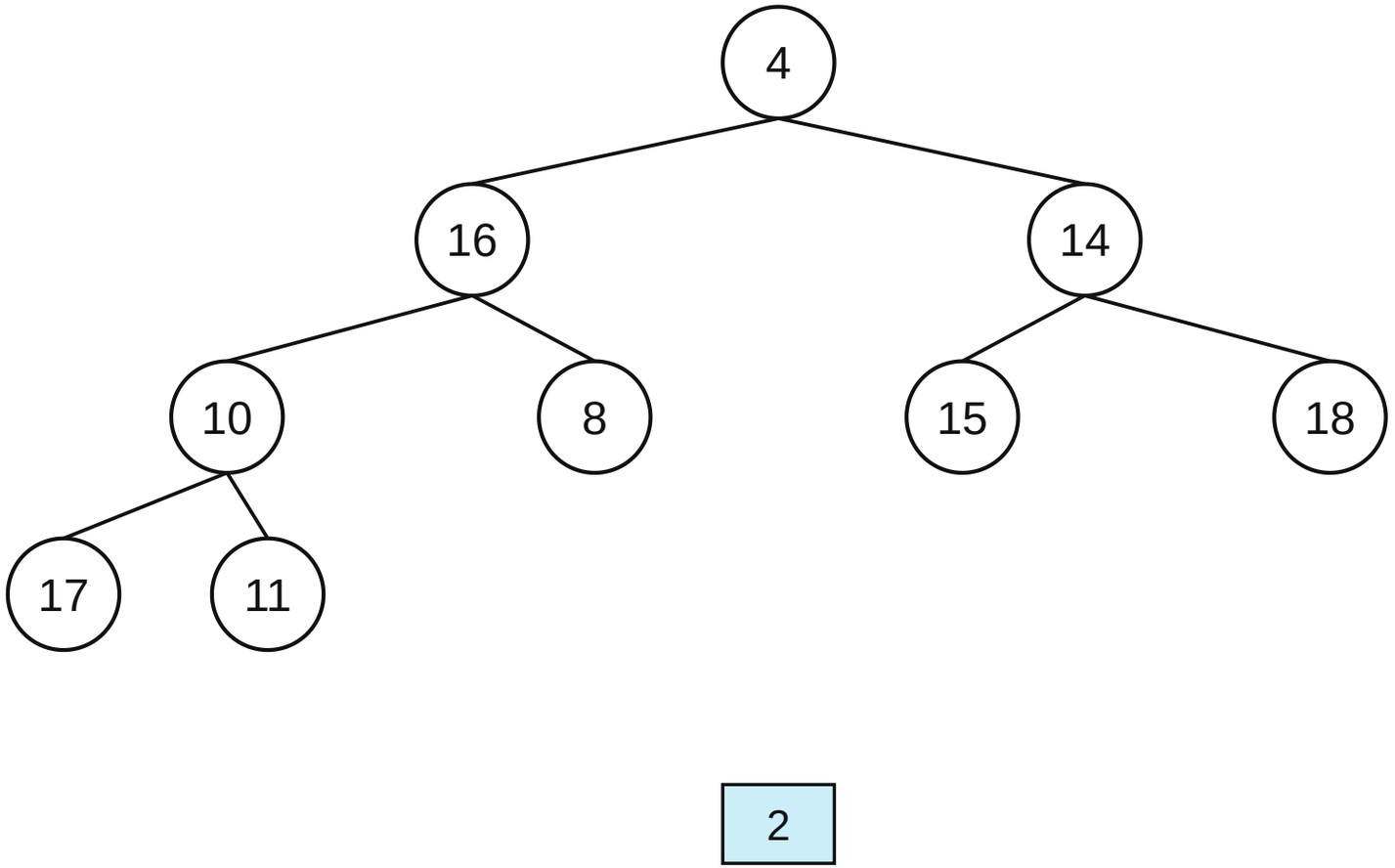
Universität Konstanz

**Move last to top**



# Heapsort: Example

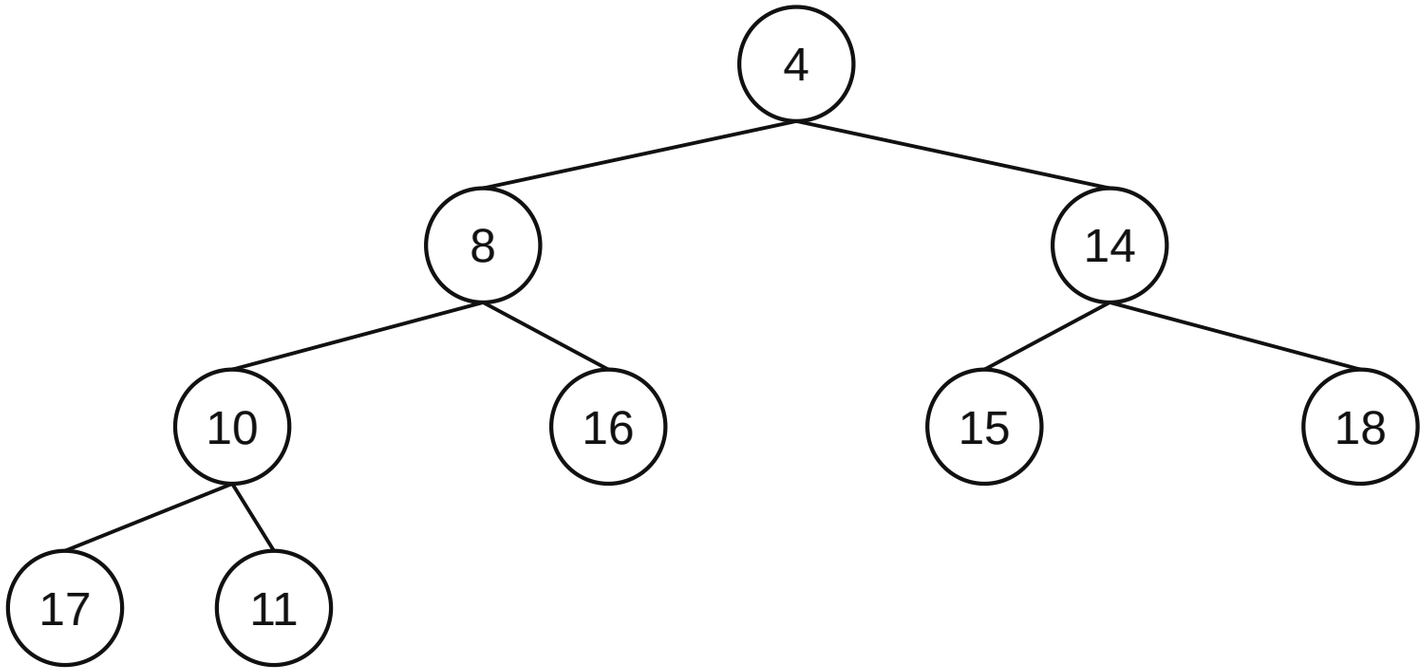





**Recover min-heap property**

# Heapsort: Example





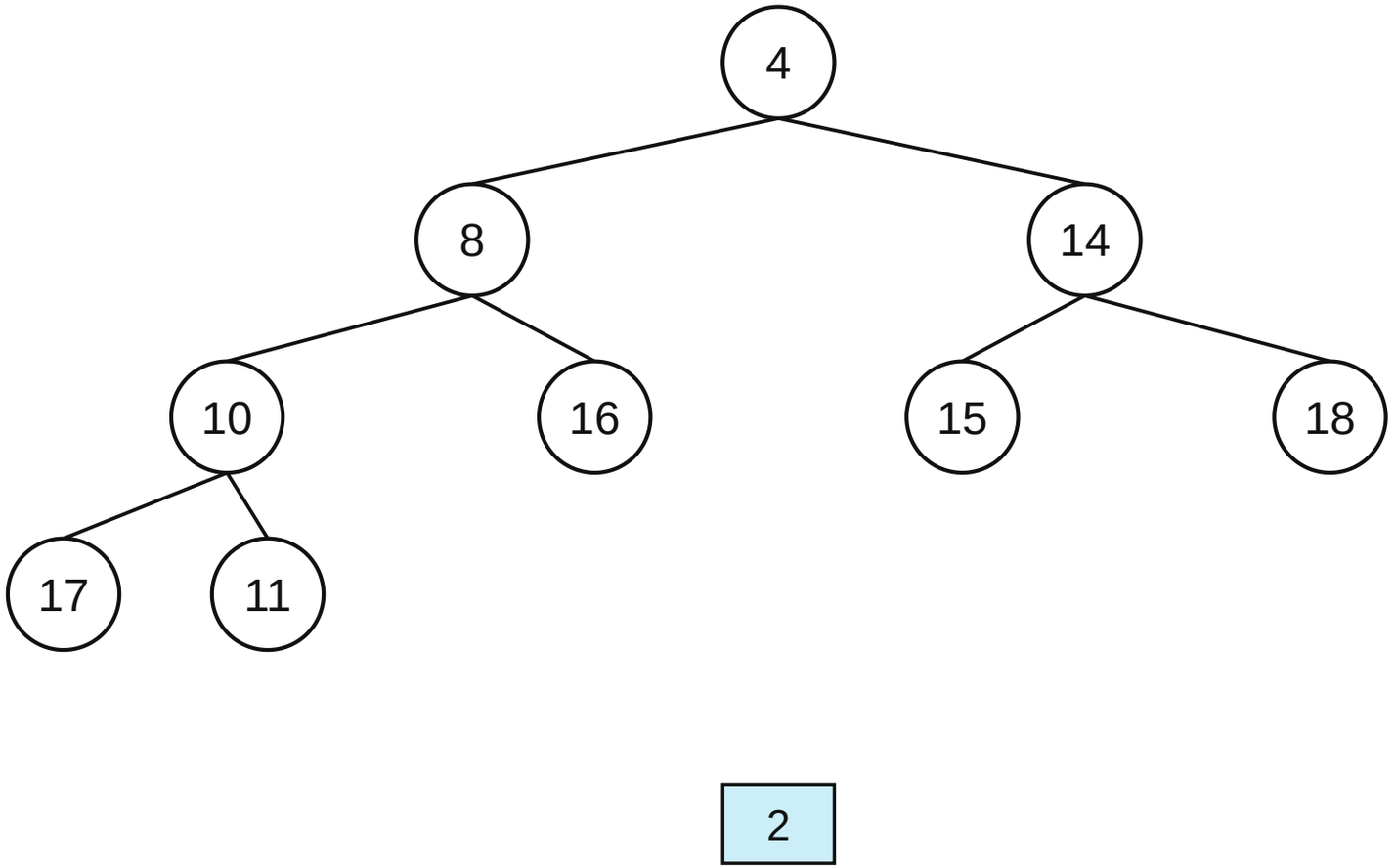

**Recover min-heap property**

2

# Heapsort: Example

Universität Konstanz

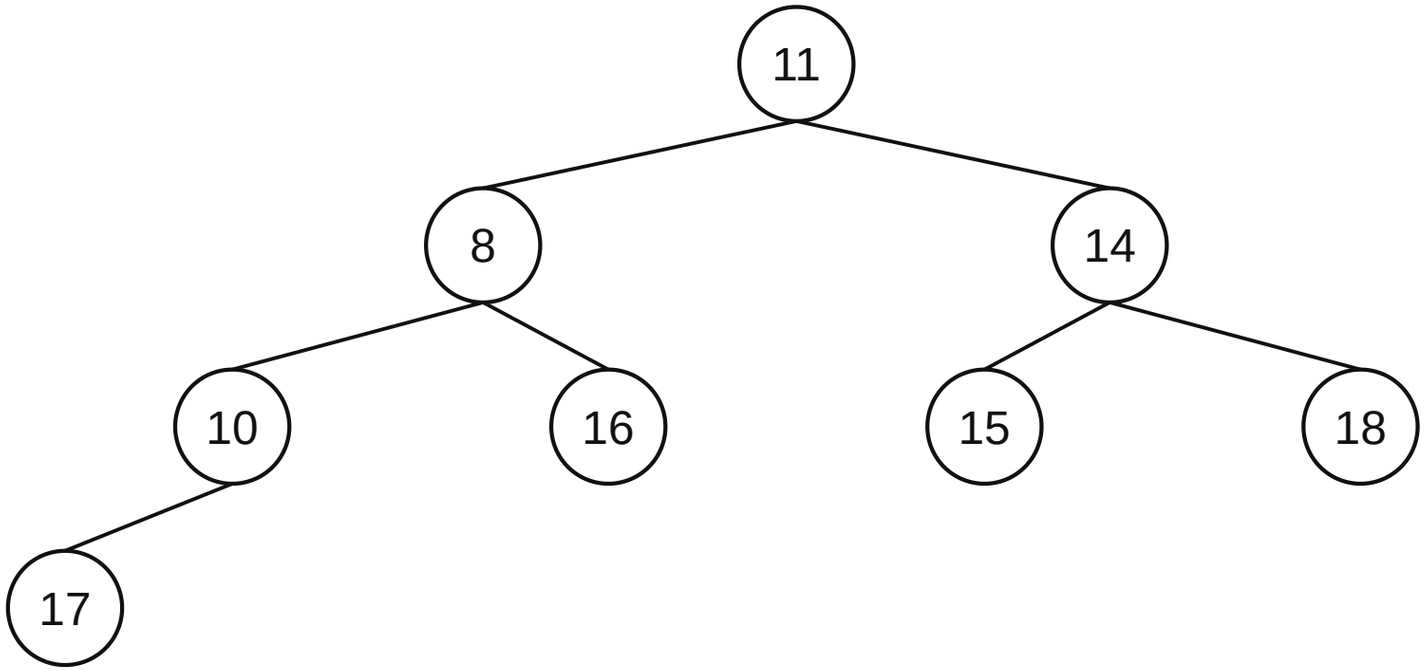
**Extract min**



# Heapsort: Example

Universität Konstanz

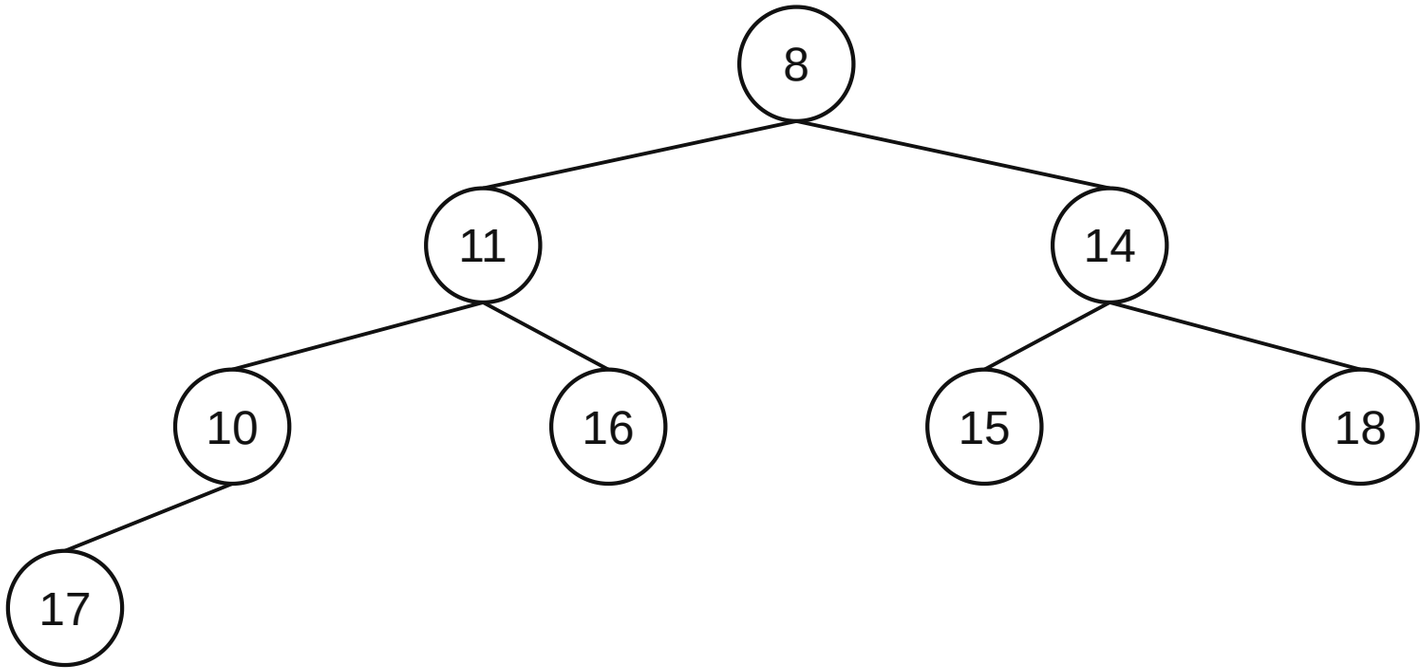
**Move last to top**




| 2 | 4 |
|---|---|

# Heapsort: Example



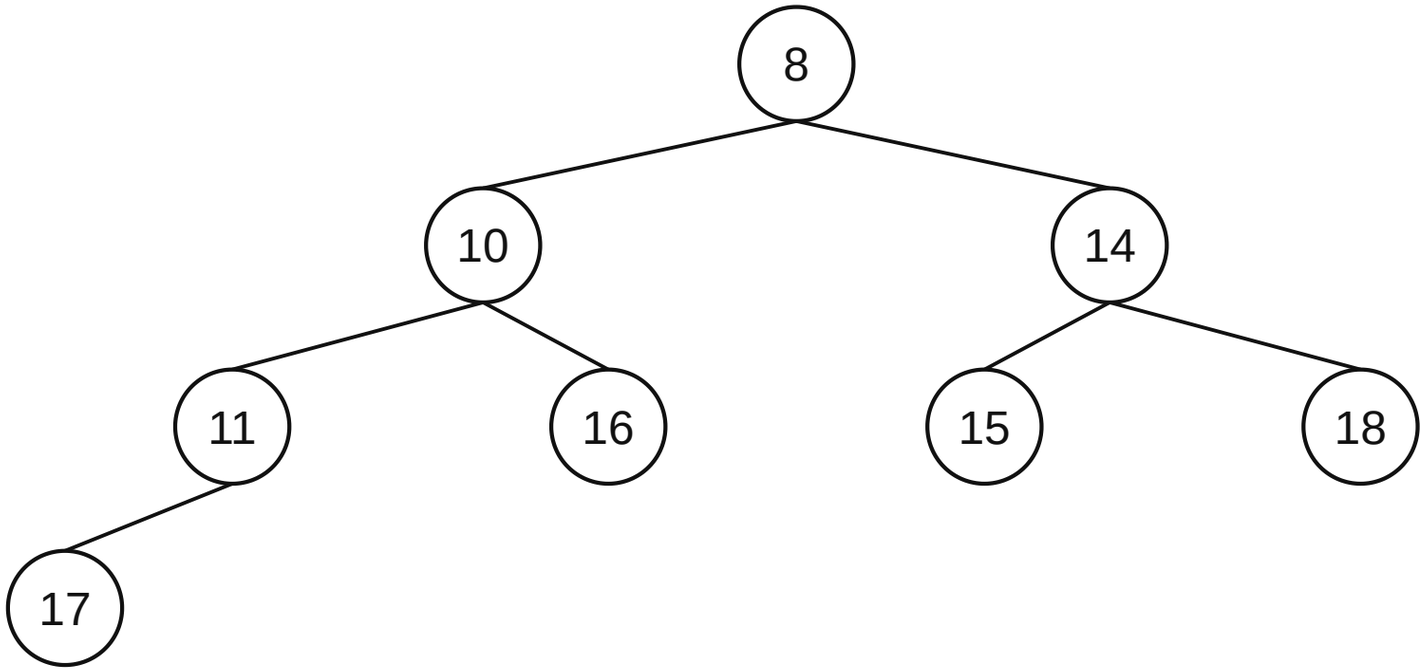



| 2 | 4 |
|---|---|

**Recover min-heap property**

# Heapsort: Example





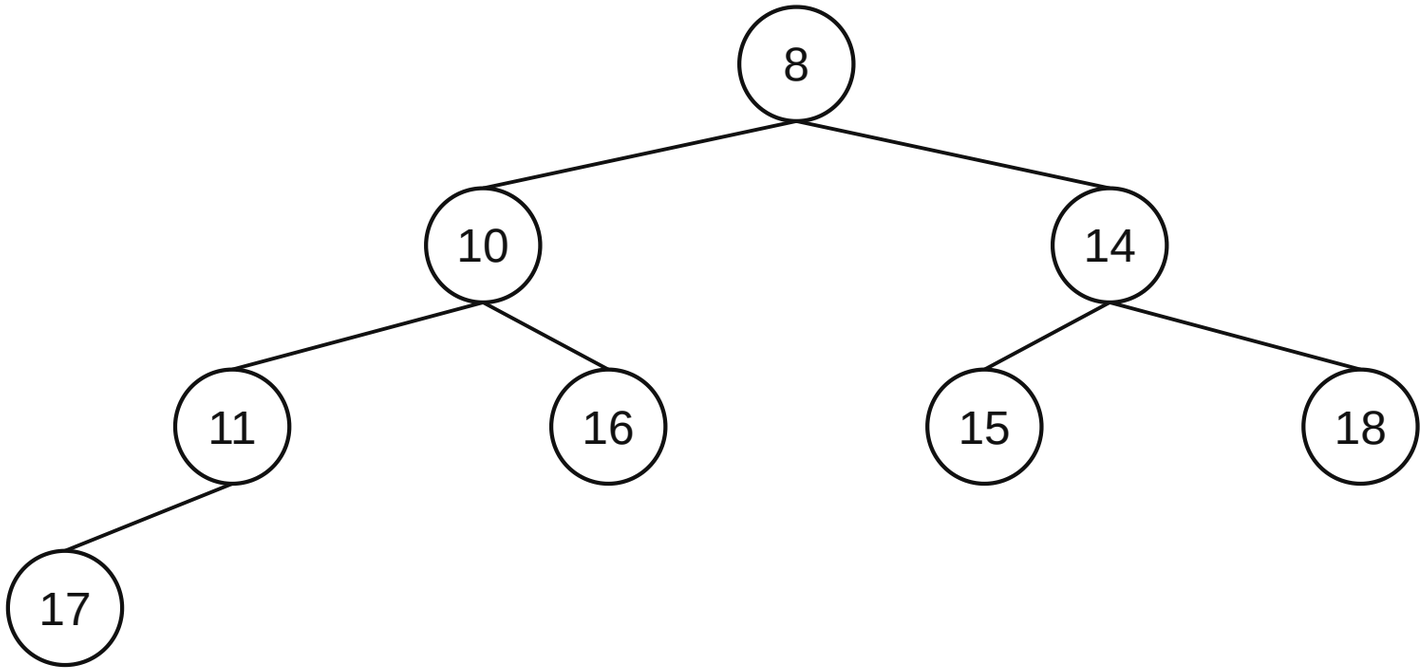

| 2 | 4 |
|---|---|

**Recover min-heap property**

# Heapsort: Example

Universität Konstanz

**Extract min**

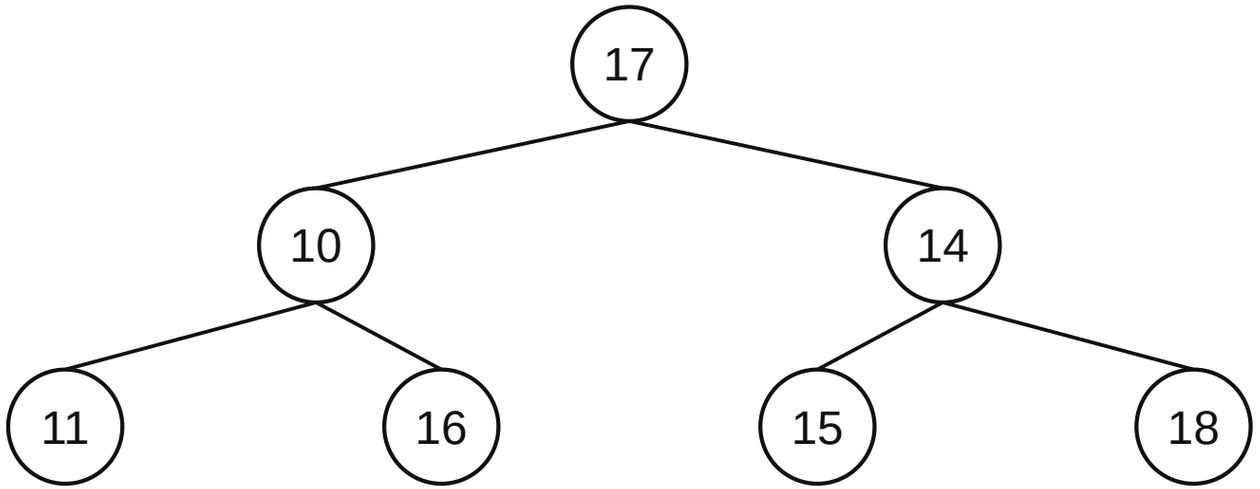



| 2 | 4 |
|---|---|

# Heapsort: Example



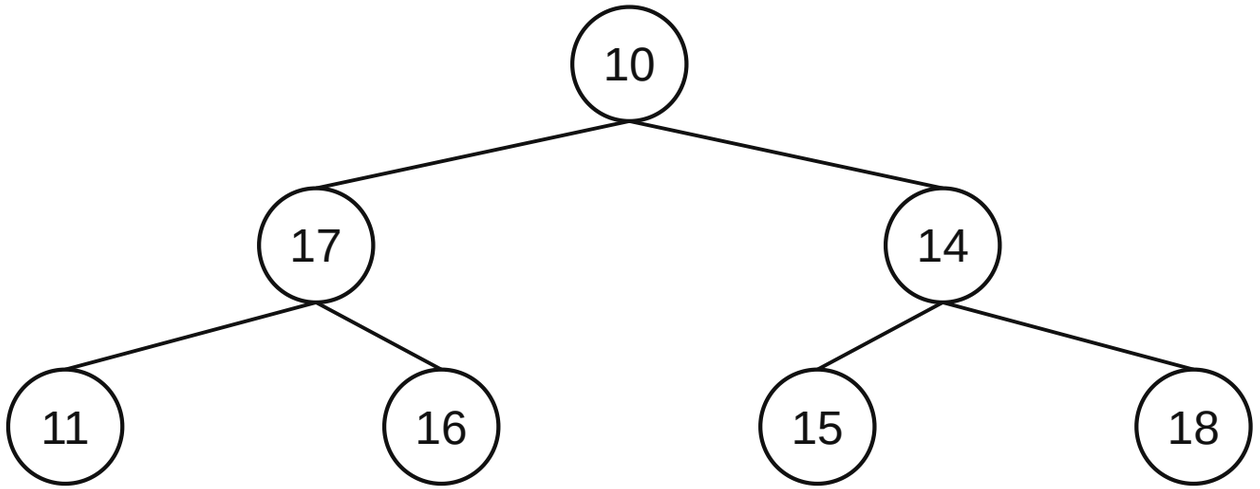
**Move last to top**




| 2 | 4 | 8 |
|---|---|---|

# Heapsort: Example



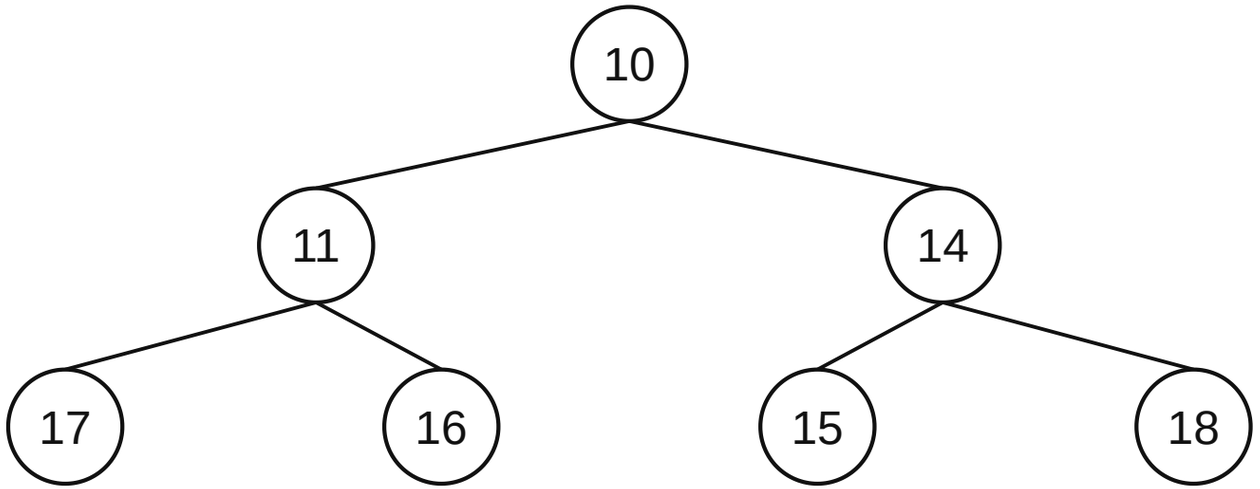



**Recover min-heap property**

| 2 | 4 | 8 |
|---|---|---|

# Heapsort: Example





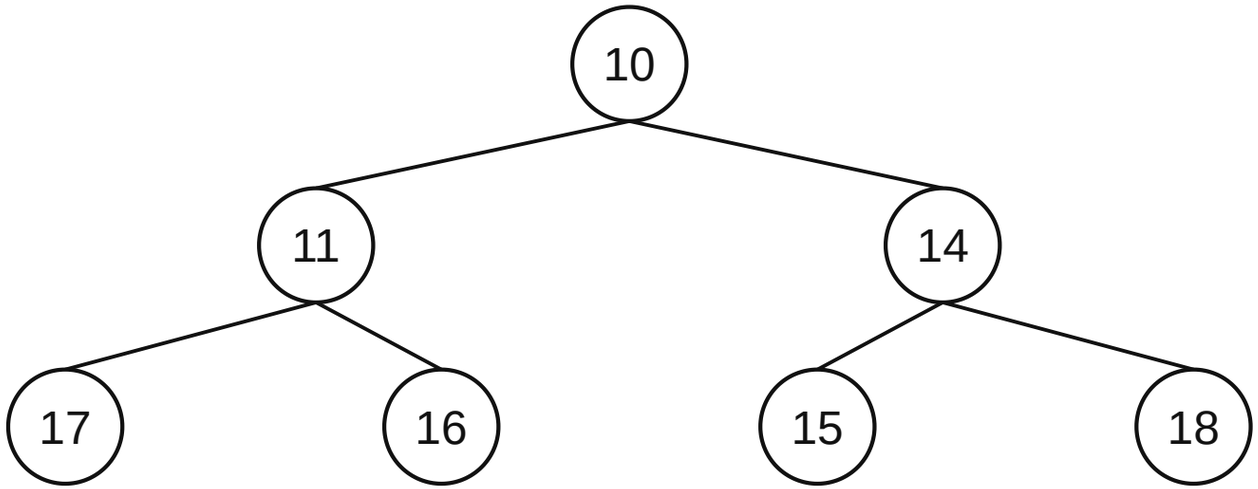

**Recover min-heap property**

| 2 | 4 | 8 |
|---|---|---|

# Heapsort: Example

Universität Konstanz

**Extract min**

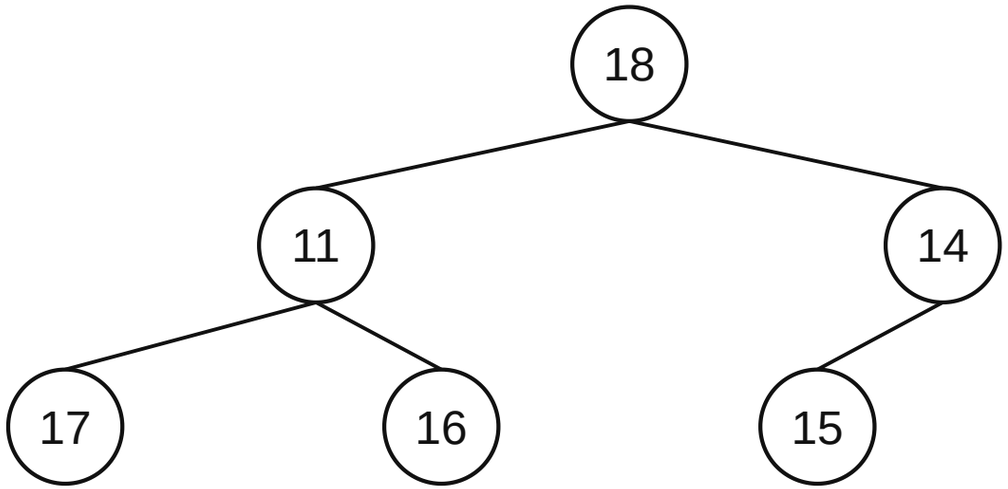



| 2 | 4 | 8 |
|---|---|---|

# Heapsort: Example



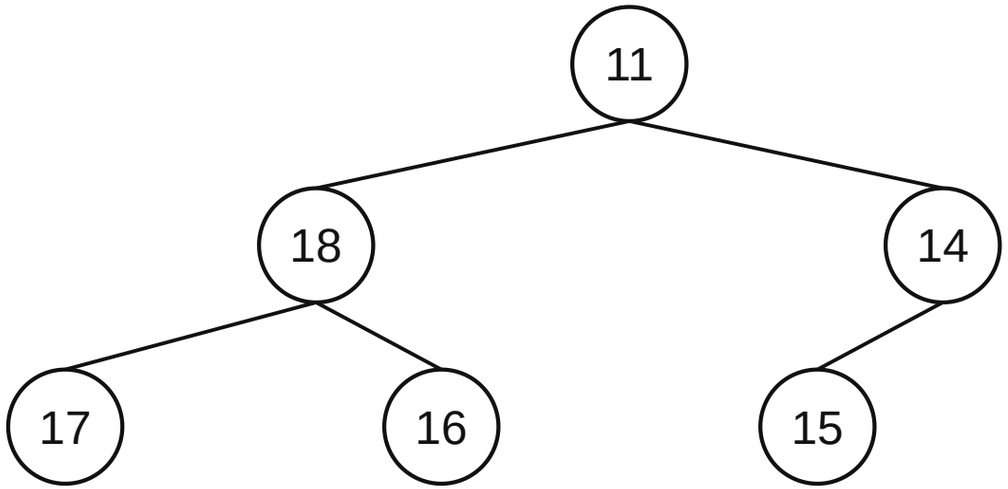
**Move last to top**




| 2 | 4 | 8 | 10 |
|---|---|---|---|

# Heapsort: Example



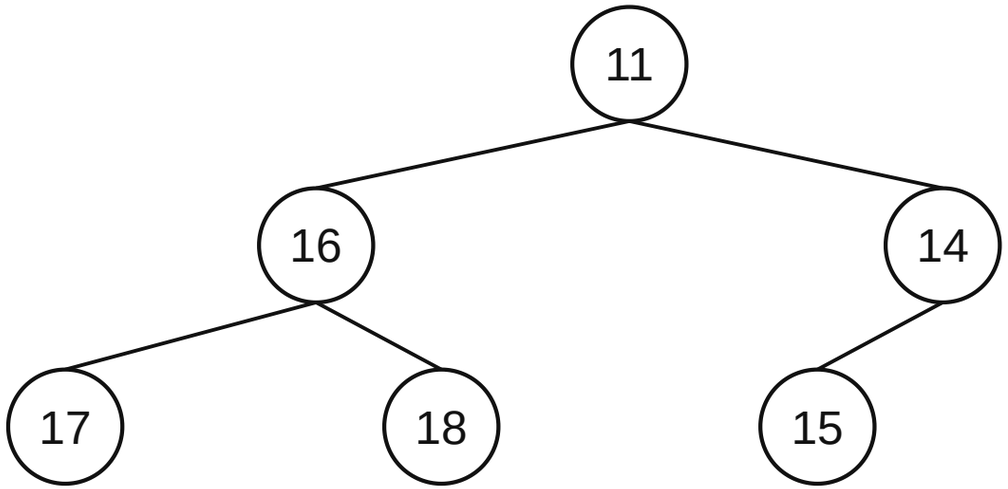



**Recover min-heap property**

| 2 | 4 | 8 | 10 |
|---|---|---|---|

# Heapsort: Example





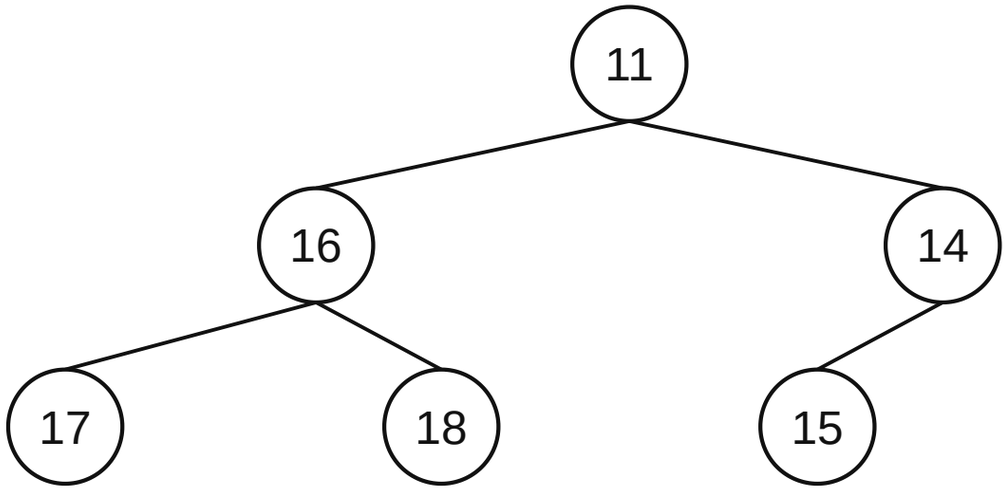

**Recover min-heap property**

| 2 | 4 | 8 | 10 |
|---|---|---|---|

# Heapsort: Example

Universität Konstanz

**Extract min**

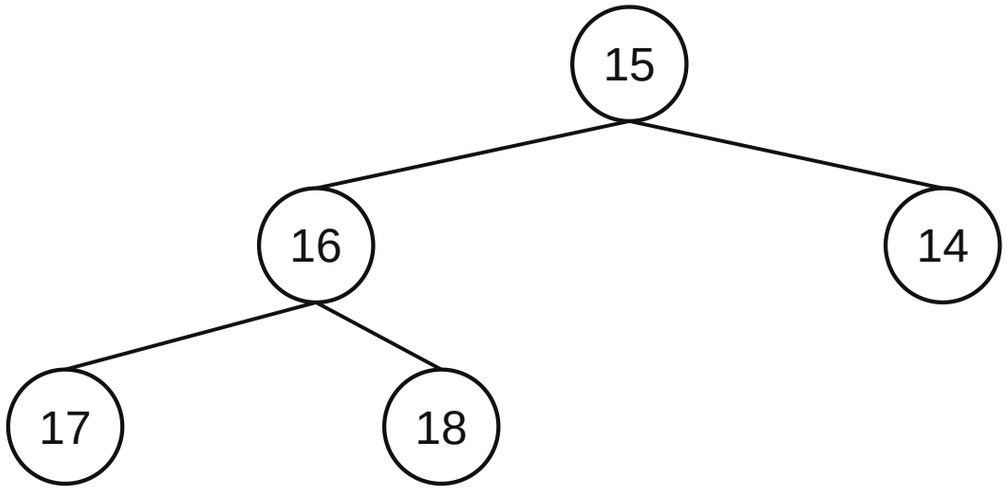



| 2 | 4 | 8 | 10 |
|---|---|---|---|

# Heapsort: Example



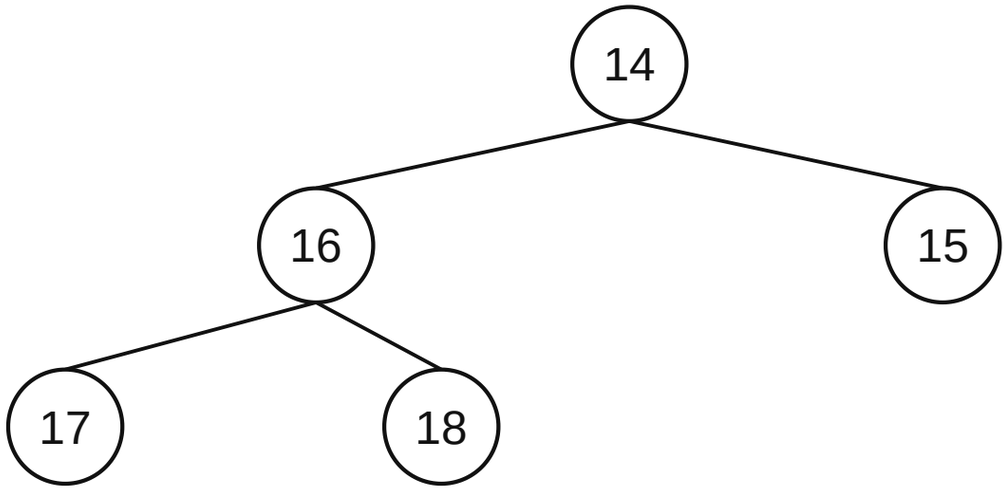
**Move last to top**




| 2 | 4 | 8 | 10 | 11 |
|---|---|---|---|---|

# Heapsort: Example





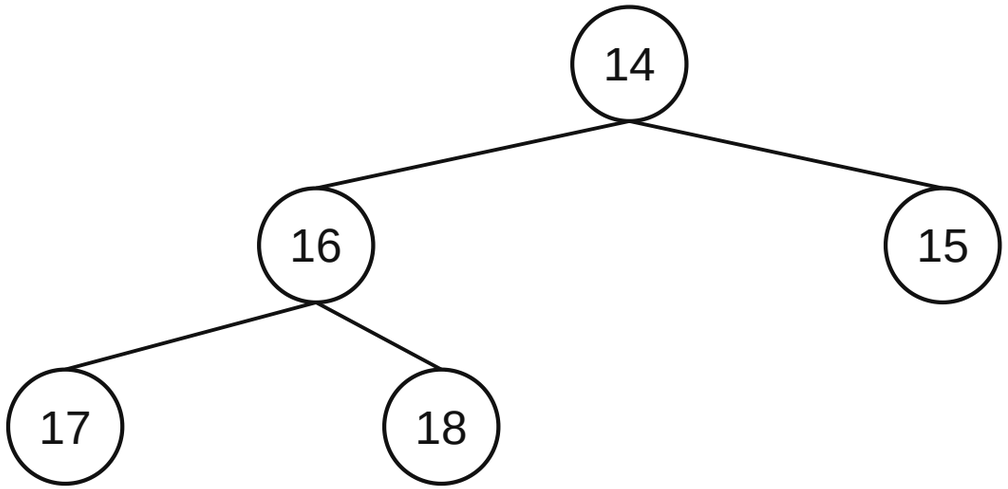

**Recover min-heap property**

| 2 | 4 | 8 | 10 | 11 |
|---|---|---|---|---|

# Heapsort: Example

Universität Konstanz

**Extract min**

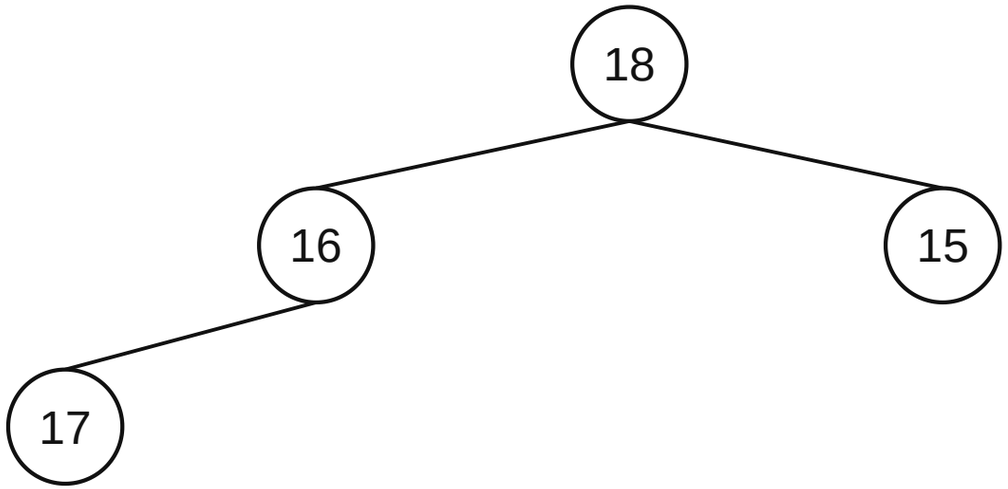



| 2 | 4 | 8 | 10 | 11 |
|---|---|---|---|---|

# Heapsort: Example



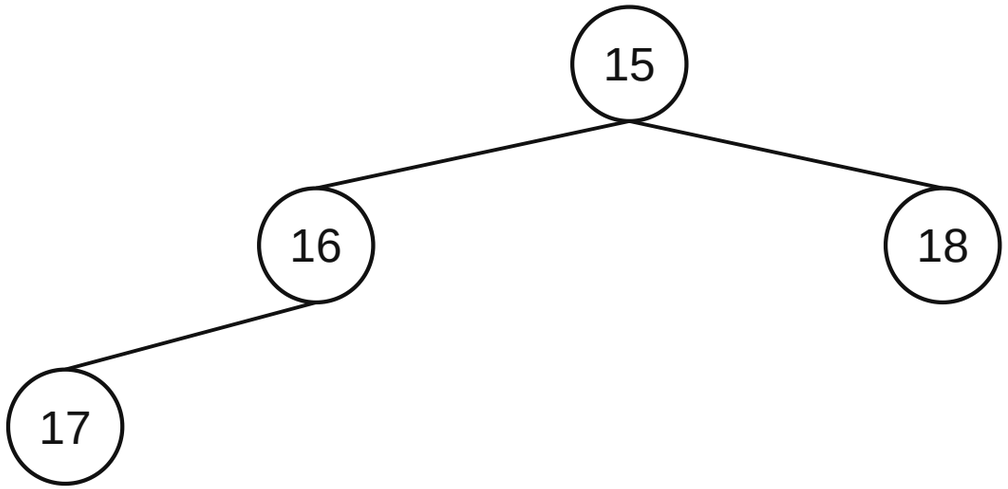
**Move last to top**




| 2 | 4 | 8 | 10 | 11 | 14 |
|---|---|---|---|---|---|

# Heapsort: Example





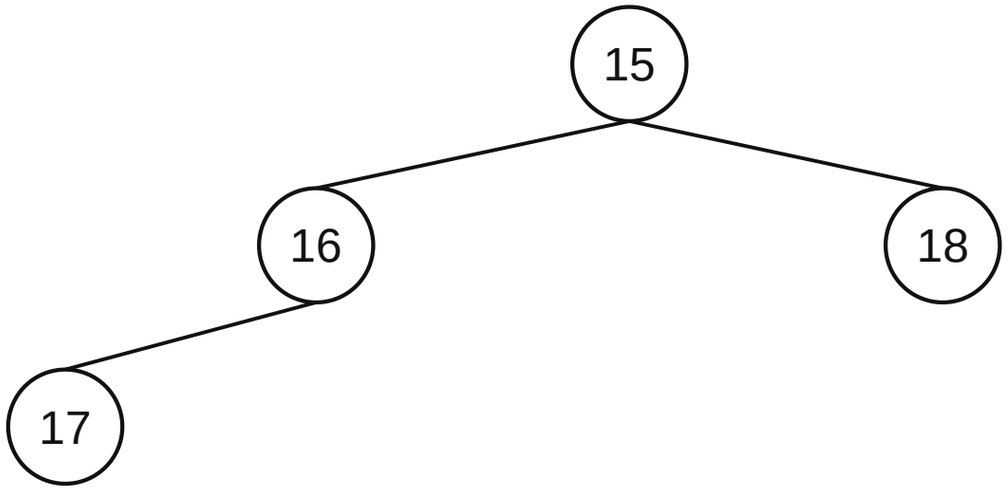

**Recover min-heap property**

| 2 | 4 | 8 | 10 | 11 | 14 |
|---|---|---|---|---|---|

# Heapsort: Example

Universität Konstanz

**Extract min**

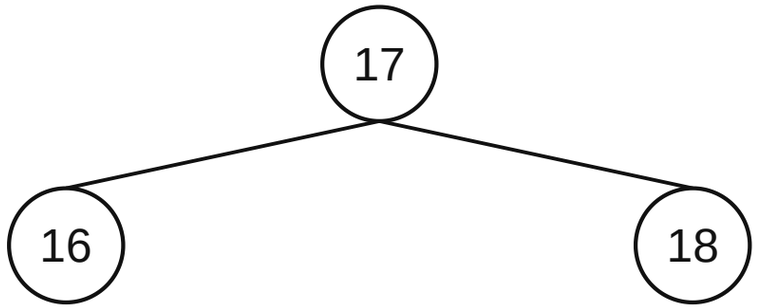



| 2 | 4 | 8 | 10 | 11 | 14 |
|---|---|---|---|---|---|

# Heapsort: Example



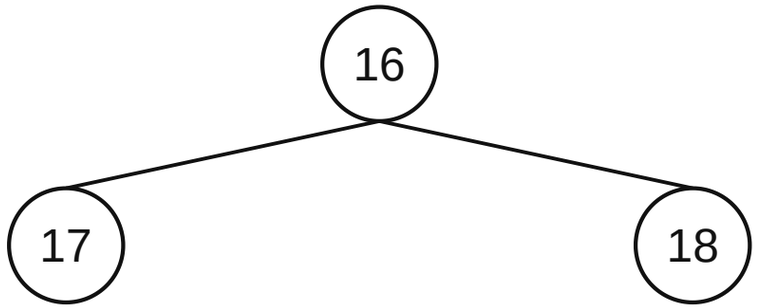
**Move last to top**




| 2 | 4 | 8 | 10 | 11 | 14 | 15 |
|---|---|---|---|---|---|---|

# Heapsort: Example





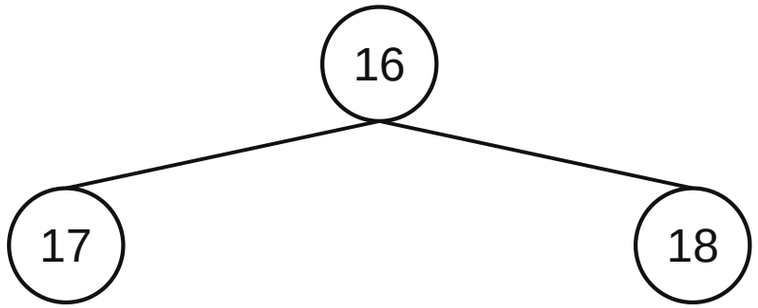

**Recover min-heap property**

| 2 | 4 | 8 | 10 | 11 | 14 | 15 |
|---|---|---|---|---|---|---|

# Heapsort: Example

Universität Konstanz

**Extract min**

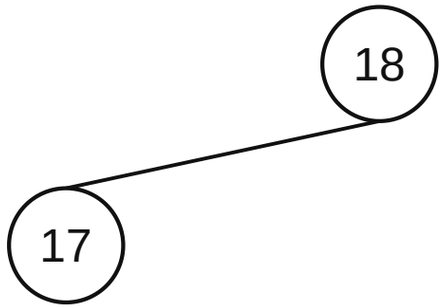



| 2 | 4 | 8 | 10 | 11 | 14 | 15 |
|---|---|---|---|---|---|---|

# Heapsort: Example



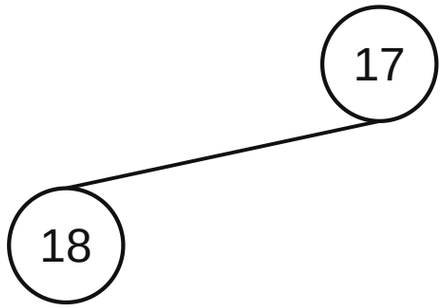
**Move last to top**




| 2 | 4 | 8 | 10 | 11 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|

# Heapsort: Example





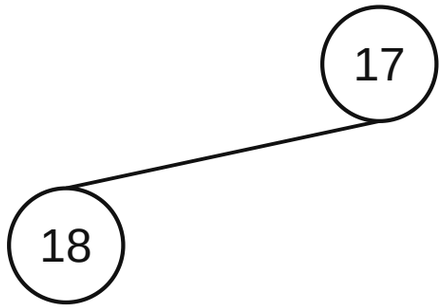

**Recover min-heap property**

| 2 | 4 | 8 | 10 | 11 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|

# Heapsort: Example

Universität Konstanz

**Extract min**




| 2 | 4 | 8 | 10 | 11 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|

# Heapsort: Example

Universität Konstanz

**Move last to top**




| 2 | 4 | 8 | 10 | 11 | 14 | 15 | 16 | 17 |
|---|---|---|---|---|---|---|---|---|

# Heapsort: Example

Universität Konstanz

**Extract min**




| 2 | 4 | 8 | 10 | 11 | 14 | 15 | 16 | 17 |
|---|---|---|---|---|---|---|---|---|

# Heapsort: Pseudocode

*Procedure* HEAPSORT$(A, n)$

*Inputs:*

- $A$: an array.
- $n$: the number of elements in $A$ to sort.

*Output:* An array $B$ containing the elements of $A$, sorted.

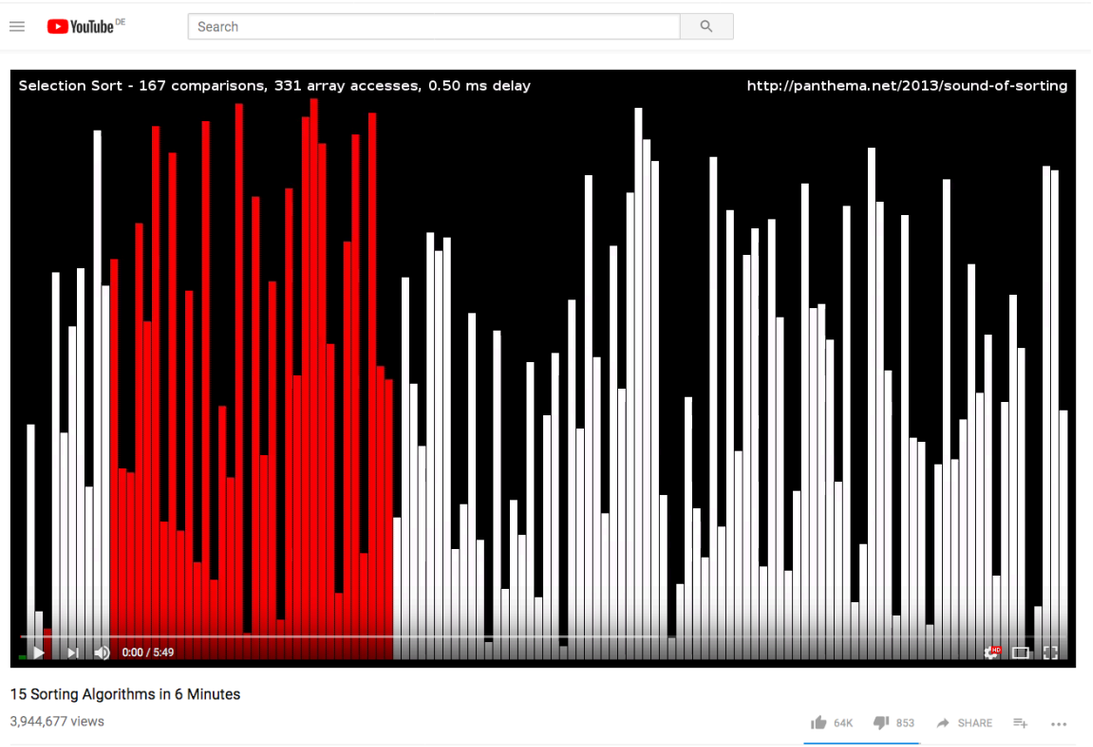
1. Build a binary heap $Q$ from the elements of $A$.
2. Let $B[1\,..\,n]$ be a new array.
3. For $i = 1$ to $n$:
   A. Call EXTRACT-MIN$(Q)$ and set $B[i]$ to the value returned.
4. Return the $B$ array.

See *Algorithms Unlocked* (Cormen, 2013, pp. 98–99)

**Exercise: what is the pseudocode to build a heap and of EXTRACT-MIN? Is Heapsort stable?**

# To learn more about sorting

**Procedure** I-CANT-BELIEVE-IT-CAN-SORT(*A*, *n*)
*Inputs:*
*A*: an array
*n*: the number of elements in *A* to sort
*Result:* the elements of *A* are sorted
into nondecreasing order
**For** *i* = 1 to *n*:
    **For** *j* = 1 to *n*:
        **If** *A*[*i*] < *A*[*j*]
            **then** swap *A*[*i*] with *A*[*j*]




https://www.youtube.com/watch?v=kPRA0W1kECg

*Is this the simplest (and most surprising) sorting algorithm ever?* Stanley Fung, 2021

# Summary



- A **stable** sort keeps equal keys in their original order
- **Selection sort** needs about $n^2/2$ comparisons
- **Mergesort** splits in halves and merges: about $n \log_2 n$ comparisons
- **Quicksort** partitions around a pivot: fast on average, $n^2$ at worst
- **Heapsort** extracts the minimum of a **min-heap** $n$ times
- Sorted data can be searched and merged much faster

**Next session:** Time Complexity